PLATAFORMA ACADÉMICA DISTRIBUIDA

Sistema académico distribuido para colegios públicos de Educación Secundaria

Institución: Escuela Superior La Pontificia
Carrera: Ingeniería de Sistemas de Información
Ciclo: VIII – B
Curso: PROYECTO DE SOFTWARE INTEGRADO DE SISTEMAS DISTRIBUIDOS, GESTIÓN DE BASE DE DATOS Y ARQUITECTURA DE SOFTWARE
Proyecto: Sistema académico distribuido para colegios (PAD)
Docente: Ing. Erick Jhonatan Palomino Alanya

Integrantes:
Baez Chipana, Jhon Ronald
Parco Bellido, Erikson
Quicaño Moreno, Cristhian Junior
Gamboa Quispe, Jhosetd Deyker
Curahua Morales, Any Miluska
Quispe Sosa, Jheremin Maysue

Entrega: Informe final – Semana 15

Ayacucho, 2026

Índice

Introducción **3**

1 Descripción del Proyecto **4**

1.1 Objetivos 5

1.2 Justificación 6

2 Proceso de Desarrollo del Proyecto **7**

2.1 Definición y Especificación del Proyecto 7

2.1.1 Definición General del Proyecto 7

2.1.2 Especificaciones de Análisis y Requerimientos del Sistema 8

2.1.3 Plan del Proyecto (Diagrama de Gantt) 12

2.2 Arquitectura del Sistema 12

2.2.1 Descripción Jerárquica 12

2.2.2 Diagrama de Módulos 13

2.2.3 Descripción Individual de los Módulos 14

2.2.4 Arquitectura de los Servicios Web y Topología de Red 15

2.2.5 Descripción del Middleware de Comunicaciones 18

2.2.6 Dependencias Externas 20

2.2.7 Fundamentos de Sistemas Distribuidos Aplicados 21

2.2.8 Decisiones de Arquitectura de Software 23

3 Diseño e Implementación **27**

3.1 Diseño del Modelo de Datos 27

3.1.1 Modelado de Proceso 27

3.1.2 Diagrama de Casos de Uso 27

3.1.3 Diagrama de Entidad-Relación 28

3.1.4 Descripción de Tablas, Llaves y Relaciones 29

3.1.5 Conexión de BD 31

3.1.6 Gestión de la Base de Datos 32

3.2 Implementación 36

3.2.1 Modelo de Interfaces 36

3.2.2 Código Fuente 43

3.2.3 Despliegue 44

4 Cierre del Proyecto **46**

4.1 Pruebas 46

4.2 Validación 47

4.3 Entrega y Puesta en Marcha 48

Conclusiones **48**

Referencias **50**

Anexos **52**

Anexo A Capturas adicionales de la interfaz 52

Anexo B Salidas de consola de las pruebas 54

Introducción

Un colegio público de Educación Secundaria gestiona cada día información crítica: la matrícula de cientos de estudiantes, la asignación de la carga horaria a los docentes, la asistencia de ingreso y de aula, las calificaciones por competencias, la convivencia escolar, la salud y el bienestar de los estudiantes y la comunicación con las familias. En la mayoría de instituciones estos procesos se llevan en cuadernos, hojas de cálculo aisladas y el SIAGIE, lo que genera duplicidad, demoras y poca trazabilidad.

El proyecto *Plataforma Académica Distribuida* (PAD) propone una solución web basada en microservicios, adaptada a la realidad de una institución educativa pública peruana y a la normativa del Ministerio de Educación (MINEDU). El sistema implementado está compuesto por **siete microservicios de dominio** desarrollados con Java 17 y Spring Boot 3.4 (Estudiantes, Cursos, Evaluaciones, Disciplina, Reportes, Notificaciones y Horarios), un **API Gateway** con Spring Cloud Gateway que centraliza la seguridad JWT, la resiliencia y los eventos en tiempo real, un intermediario de mensajería **RabbitMQ** y una instancia **PostgreSQL 15** que aloja siete bases de datos independientes, una por servicio (*database-per-service*). La interfaz web fue construida con **Angular 21** y se sirve mediante Nginx.

Todo el sistema se ejecuta con Docker Compose en once contenedores con verificación de salud (*health checks*). La interfaz está diferenciada para once roles institucionales —Director, Administrador del sistema, Coordinación académica, Coordinación de convivencia, Secretaría, Docente, Auxiliar, Psicología, Enfermería, Mantenimiento y Apoderado— y aplica separación de funciones y mínimo privilegio.

Este informe documenta el alcance funcional, la arquitectura, el modelo de datos distribuido y la implementación, e incluye **evidencias reales** obtenidas el 13 de octubre de 2026 sobre el sistema en ejecución: capturas de cada módulo, el estado de los contenedores, la mensajería y los resultados de las pruebas automatizadas de backend, frontend, humo (*smoke*) y contrato del gateway.

Por tratarse del proyecto integrador de los cursos de **Sistemas Distribuidos**, **Arquitectura de Software** y **Gestión de Base de Datos**, cada decisión se explica desde esas tres perspectivas. Desde Sistemas Distribuidos se analizan la comunicación entre procesos, la tolerancia a fallos, la consistencia, la idempotencia y la observabilidad (apartado 2.2.7). Desde Arquitectura de Software se justifican el estilo arquitectónico, los patrones, las vistas del

modelo 4+1, los atributos de calidad y las decisiones de diseño con sus alternativas (apartado 2.2.8). Desde Gestión de Base de Datos se fundamentan el modelo relacional, la normalización, la integridad, las transacciones, la concurrencia, las migraciones, la auditoría y la estrategia de respaldo y recuperación (apartado 3.1.6). La Tabla 1 resume esta integración.

Tabla 1

Integración de los tres cursos en el proyecto

Curso	**Conceptos aplicados**	**Dónde se implementa**
Sistemas Distribuidos	Procesos independientes que se comunican por red; comunicación síncrona (REST) y asíncrona (mensajería); transparencia de ubicación; tolerancia a fallos; consistencia eventual; idempotencia; trazabilidad distribuida.	Siete microservicios y gateway en contenedores; RabbitMQ con DLQ; *circuit breaker*; patrón Outbox; SSE; ID de correlación y Zipkin.
Arquitectura de Software	Estilo de microservicios con capas internas; contextos delimitados; patrones API Gateway, *Database per Service*, Outbox y *Circuit Breaker*; vistas 4+1; atributos de calidad ISO/IEC 25010; registro de decisiones.	Figuras 2 a 9; Tablas de patrones, atributos de calidad y decisiones del apartado 2.2.8.
Gestión de Base de Datos	Modelo relacional normalizado; integridad de entidad, referencial y de dominio; índices; transacciones ACID; control de concurrencia; migraciones versionadas; auditoría; respaldo cifrado y restauración verificada.	Siete bases PostgreSQL con 55 tablas, 32 claves foráneas, 24 restricciones UNIQUE y 19 CHECK; Flyway; Envers; servicio de respaldo.

Nota. Elaboración propia.

1. Descripción del Proyecto

La Plataforma Académica Distribuida es un sistema web para la gestión académica y de convivencia de un colegio público de Educación Secundaria (I.E.P. Huamanga, institución de demostración), con datos de prueba de quince secciones de primero a quinto grado en turnos mañana y tarde, 500 estudiantes y 30 docentes. Automatiza la matrícula y la trayectoria del estudiante, la carga docente y los horarios, el ingreso al colegio con DNI, la asistencia, el registro de notas por competencias con la escala del Currículo Nacional (AD, A, B, C), la promoción de grado y la recuperación pedagógica, la convivencia escolar, la salud, el bienestar y las notificaciones a las familias.

La distribución se realiza por dominios de negocio. Ningún servicio consulta directamente las tablas de otro; cuando necesita confirmar la carga académica de un docente, los datos de un estudiante o enviar una notificación, utiliza la API REST interna del servicio propietario, protegida además con una clave interna (`INTERNAL_API_KEY`). Los cambios relevantes se difunden en tiempo real a todas las pantallas abiertas mediante *Server-Sent Events* (SSE) emitidos por el gateway.

1.1 Objetivos

Objetivo general. Diseñar e implementar una plataforma académica distribuida para colegios públicos que gestione la matrícula, la carga docente y los horarios, la asistencia, las calificaciones por competencias, la convivencia y la comunicación con las familias mediante microservicios independientes, bases PostgreSQL aisladas, un API Gateway seguro, mensajería asíncrona y una interfaz web intuitiva que funcione en tiempo real.

Objetivos específicos.

1. Implementar autenticación sin estado con JWT y autorización por once roles institucionales, validada en el API Gateway y repetida en los servicios sensibles.
2. Registrar la matrícula gratuita (nuevo ingreso, continuidad y traslado) conforme a la normativa del MINEDU, conservando la trayectoria del estudiante por año escolar.
3. Gestionar niveles, grados, secciones, cursos, docentes, carga académica y horarios, con un generador de horarios parametrizable y editable, y reparto de un curso entre varios docentes.
4. Registrar el ingreso al colegio escaneando el código de barras del DNI (lector USB o cámara), con detección automática de tardanzas.
5. Registrar asistencia por sesión de clase y calificaciones por competencias con escala AD/A/B/C, incluido el dictado de notas por voz.
6. Gestionar el cierre de bimestres, la promoción de grado, las áreas por subsanar y la recuperación pedagógica.
7. Gestionar convivencia escolar, derivaciones psicológicas, fichas médicas, atenciones de tópico y tickets de mantenimiento.
8. Notificar a las familias mediante el patrón *Outbox* y RabbitMQ con reintentos y cola de mensajes fallidos (DLQ).

9. Proporcionar indicadores, alertas de deserción, auditoría con Hibernate Envers y reconciliación de integridad entre servicios.
10. Contenerizar todo el sistema con Docker Compose y comprobarlo con pruebas automatizadas (Testcontainers, Vitest, smoke y contrato).

1.2 Justificación

El manejo manual de matrículas, asistencia y notas provoca duplicidad, pérdida de información y dificultad para obtener reportes oportunos para la UGEL y para las familias. Además, en una institución pública intervienen muchos actores con responsabilidades distintas: el Director no debe registrar notas, la Secretaría no debe ver fichas médicas y el docente solo debe acceder a sus secciones. Una plataforma web única con roles bien delimitados mejora el servicio, mientras que la arquitectura distribuida separa las responsabilidades técnicas y reduce el acoplamiento (Newman, 2021).

El patrón *database-per-service* evita que un módulo modifique datos que no le pertenecen (Richardson, 2018): Estudiantes conserva la identidad, la matrícula y el bienestar; Cursos controla la estructura curricular y la carga; Evaluaciones es responsable de notas, asistencia y bimestres; Disciplina gestiona la convivencia; Notificaciones entrega los mensajes; Reportes consolida indicadores; y Horarios ejecuta la optimización de horarios. Si un servicio falla, el gateway responde con un mensaje controlado (*circuit breaker*) y el resto del colegio sigue funcionando.

El sistema se adapta a la normativa vigente: la matrícula en instituciones públicas es gratuita y no puede condicionarse al pago de APAFA, uniformes o útiles; la calificación del Currículo Nacional de la Educación Básica (CNEB) usa la escala literal AD, A, B y C; y la RVM N.° 094-2020-MINEDU regula la promoción, la permanencia y la recuperación pedagógica (Ministerio de Educación del Perú [MINEDU], 2016, 2020). El proyecto integra además conceptos de sistemas distribuidos —APIs REST, mensajería, consistencia eventual, tolerancia a fallos y observabilidad (Kleppmann, 2017)— en un caso real y verificable.

2. Proceso de Desarrollo del Proyecto

2.1 Definición y Especificación del Proyecto

2.1.1 Definición General del Proyecto

El proyecto adopta una arquitectura de microservicios detrás de un API Gateway. La Tabla 2 resume los componentes desplegados y su puerto interno en la red Docker `pad-network`. Solo el frontend (puerto 80) y, para administración local, el gateway, PostgreSQL y la consola de RabbitMQ (ligados a 127.0.0.1) se publican fuera de la red.

Tabla 2

Componentes del sistema desplegado

Componente	Puerto	Responsabilidad
Frontend Angular + Nginx	80	Interfaz web por roles; Nginx sirve la aplicación y reenvía `/api` al gateway.
API Gateway	8080	Enrutamiento, validación JWT y RBAC, CORS, *circuit breaker* con respuestas de respaldo, ID de correlación y eventos en tiempo real (SSE).
servicio-estudiantes	8082	Usuarios y autenticación, estudiantes, apoderados, matrícula y trayectoria, traslados, ingreso por DNI, recuperación, documentos, consentimientos, fichas médicas, tópico, derivaciones y portal del apoderado.
servicio-cursos	8083	Niveles, grados, secciones, periodos escolares, cursos, docentes, aulas, carga académica, reparto de carga, horarios y tickets de mantenimiento.
servicio-evaluaciones	8084	Notas por competencias (AD/A/B/C), sesiones y registros de asistencia, estado y auditoría de bimestres.
servicio-disciplina	8081	Reportes de convivencia escolar y notificación al apoderado.
servicio-reportes	8085	Indicadores del inicio, alertas de deserción, actividad reciente y reconciliación de integridad entre servicios.
servicio-notificaciones	8086	Recepción de eventos, cola RabbitMQ, entrega por correo/SMS/WhatsApp con reintentos y DLQ.
servicio-horarios	8088	Parámetros y motor de optimización de horarios con Timefold Solver.
PostgreSQL 15	5432	Siete bases independientes, una por servicio, versionadas con Flyway.
RabbitMQ 4.3	5672 / 15672	Intercambio `pad.eventos`, cola `pad.notificaciones` y cola de fallidos `pad.notificaciones.dlq`.

Nota. Además existen perfiles opcionales de Docker Compose: `observability` (Prometheus, Grafana y Zipkin), `operations` (respaldo de PostgreSQL), `demo` (carga de datos de secundaria 2026) y pgAdmin. Elaboración propia a partir de `docker-compose.yml`.

Cada servicio Spring Boot sigue una separación por capas: controlador REST, DTO, servicio de aplicación, entidad JPA, repositorio y configuración transversal (seguridad, manejador global de errores, OpenAPI y migraciones Flyway). El alcance implementado cubre el ciclo completo del año escolar, desde la matrícula hasta la promoción y la recuperación pedagógica.

2.1.2 Especificaciones de Análisis y Requerimientos del Sistema

Actores y necesidades. La Tabla 3 presenta la matriz de roles del sistema, aplicada con separación de funciones y mínimo privilegio: el Director conserva visibilidad y gobierno, pero las operaciones diarias se distribuyen entre el personal que normalmente las ejecuta. Una persona puede reunir varios roles y las autorizaciones usan la unión de ellos.

Tabla 3

Matriz de roles institucionales

Rol	Responsabilidad	Funciones habilitadas
`DIRECTOR`	Gobierno institucional	Indicadores, reportes, auditoría, consentimientos, seguimiento de notificaciones y reapertura justificada de bimestres.
`ADMIN_SISTEMA`	Administración técnica	Crear usuarios, asignar roles, activar o desactivar cuentas y restablecer accesos.
`COORDINADOR_ACADEMICO`	Gestión pedagógica	Cursos, secciones, carga docente, reparto de carga, horarios, supervisión de notas y cierre de bimestres.
`COORDINADOR_CONVIVENCIA`	Convivencia escolar	Asistencia, disciplina, derivaciones y seguimiento de comunicaciones.
`SECRETARIA`	Gestión documental	Matrícula, traslados, estudiantes, apoderados, documentos, consentimientos, promoción y recuperación, ingreso al colegio.
`DOCENTE`	Trabajo en aula	Solo sus secciones: notas, asistencia y convivencia; derivaciones y solicitudes de mantenimiento.
`AUXILIAR`	Control cotidiano	Ingreso al colegio con DNI, asistencia y disciplina.
`PSICOLOGO`	Bienestar socioemocional	Consulta y atención de derivaciones.
`ENFERMERA`	Salud escolar	Fichas médicas y atenciones de tópico.

Rol	Responsabilidad	Funciones habilitadas
`MANTENIMIENTO`	Infraestructura	Consulta de aulas, seguimiento y resolución de tickets.
`APODERADO`	Consulta familiar	Asistencia, notas, documentos y comunicaciones solo de sus hijos vinculados.

Nota. Elaboración propia a partir de `documentacion/MATRIZ_ROLES.md` y de las reglas de `api-gateway/SecurityConfig.java`.

Requerimientos funcionales. La Tabla 4 detalla los requerimientos funcionales implementados; la columna «Evidencia» remite a la figura o prueba que los demuestra.

Tabla 4

Requerimientos funcionales

Código	Requerimiento funcional	Evidencia
RF-01	Autenticar por usuario y contraseña, emitir un JWT con roles y exigir el cambio de contraseña en el primer ingreso.	Fig. 11; smoke
RF-02	Mostrar a cada rol solo los módulos que le corresponden y bloquear las rutas ajenas (401/403).	Fig. 12; smoke, contrato
RF-03	Registrar la matrícula gratuita como nuevo ingreso, continuidad o traslado, exigiendo solo el DNI o acta de nacimiento.	Fig. 16
RF-04	Conservar la trayectoria del estudiante por año escolar y registrar el traslado de salida con IE de destino.	`TrayectoriaIngreso-IntegrationTest`
RF-05	Gestionar niveles, grados, secciones, turnos, cursos, docentes, aulas y periodos escolares.	Fig. 14; Fig. A2
RF-06	Asignar la carga académica y repartir un mismo curso entre varios docentes respetando sus horas contratadas.	`RepartoCarga-IntegrationTest`
RF-07	Generar horarios a partir de parámetros (turno, hora de inicio, minutos por periodo, días, recreos y reglas) y editarlos arrastrando bloques.	Fig. 14, 15
RF-08	Rechazar cruces de docente o sección y el exceso de carga al guardar horarios.	`HorarioLote-IntegrationTest`
RF-09	Registrar el ingreso al colegio escaneando el DNI (lector o cámara), clasificar puntual o tardanza y avisar al apoderado.	Fig. 19
RF-10	Registrar asistencia por sesión de clase solo para las secciones del docente.	Fig. 20
RF-11	Registrar notas por competencias con la escala AD/A/B/C, por teclado o por voz.	Fig. 17, 18
RF-12	Cerrar bimestres (Coordinación) y reabrirlos solo con justificación (Dirección), con auditoría.	Contrato

Código	Requerimiento funcional	Evidencia
RF-13	Registrar la situación final (promovido, recuperación, permanece o egresa) y las áreas por subsanar.	Fig. 22
RF-14	Gestionar la recuperación pedagógica de las áreas pendientes.	Fig. 23
RF-15	Registrar incidencias de convivencia y notificar al apoderado.	Smoke
RF-16	Gestionar fichas médicas, atenciones de tópico y derivaciones psicológicas con acceso restringido.	Smoke
RF-17	Gestionar tickets de mantenimiento creados por docentes y resueltos por Mantenimiento.	Smoke
RF-18	Enviar notificaciones con reintentos, cola de fallidos y trazabilidad por estado y canal.	Fig. 26
RF-19	Mostrar indicadores, alertas de deserción, actividad reciente y una lista de tareas del día por rol.	Fig. 12, 24
RF-20	Actualizar en tiempo real las pantallas abiertas cuando otro usuario modifica datos.	`TiempoRealServiceTest`
RF-21	Consultar el historial de auditoría (quién cambió qué y cuándo) y el portal del apoderado.	Smoke
RF-22	Exportar listas y horarios (CSV/Excel) e imprimirlos.	Fig. 14, 21

Nota. Elaboración propia.

Reglas de negocio principales.

- La matrícula en una institución pública es gratuita; no se condiciona a pagos de APAFA, uniformes, útiles ni exámenes. El único documento exigible es el DNI o el acta de nacimiento.
- Un estudiante tiene una sola matrícula por año escolar; un traslado de salida cambia su situación a `TRASLADADO` y lo retira de las listas activas.
- Un docente no puede tener dos clases a la misma hora, una sección no puede tener dos cursos en el mismo periodo y la carga semanal no puede superar las horas contratadas.
- El docente solo lee y escribe datos de las secciones y cursos de su carga académica vigente, aunque conozca el identificador de otro recurso.
- Un bimestre cerrado vuelve inmutables sus notas; solo el Director puede reabrirlo y debe registrar un motivo auditado.
- El ingreso registrado después de la hora límite del turno se clasifica como tardanza; un mismo DNI no se registra dos veces el mismo día.

- Un estudiante promovido puede llevar áreas por subsanar; las áreas en recuperación quedan pendientes hasta registrar su resultado.
- Cada notificación se guarda primero en una tabla *outbox* dentro de la misma transacción del negocio; su envío se reintenta y, si se agota, pasa a la DLQ.

Requerimientos no funcionales. La Tabla 5 presenta los requerimientos no funcionales.

Tabla 5

Requerimientos no funcionales

Código	Requerimiento no funcional	Implementación
RNF-01	Todo acceso externo pasa por un único punto de entrada con rutas `/api/**` y datos JSON.	Spring Cloud Gateway
RNF-02	Autenticación sin estado con JWT HS256; firma y expiración validadas en el gateway.	JJWT + Spring Security
RNF-03	Autorización RBAC en el gateway, repetida en los servicios sensibles, y comunicación interna con clave propia.	`INTERNAL_API_KEY`
RNF-04	Cada microservicio posee su propia base y no accede al esquema de otro.	7 bases PostgreSQL
RNF-05	Ante la caída o lentitud de un servicio, el sistema responde con un mensaje controlado en lugar de bloquearse.	Resilience4j (25 s)
RNF-06	Las pantallas se actualizan en tiempo real sin recargar la página.	SSE desde el gateway
RNF-07	Los esquemas se versionan con migraciones e incluyen claves, restricciones e índices; Hibernate solo valida.	Flyway, `ddl-auto=validate`
RNF-08	Errores con códigos HTTP correctos (4xx/5xx) y cuerpo JSON uniforme.	`GlobalExceptionHandler`
RNF-09	La infraestructura se reproduce con un solo comando y variables de entorno; los secretos no se versionan.	Docker Compose, `.env`
RNF-10	Interfaz responsiva, intuitiva, con modo claro/oscuro y accesos rápidos (Ctrl + K).	Angular 21
RNF-11	Trazabilidad: auditoría de cambios, ID de correlación por solicitud, métricas y trazas.	Envers, Micrometer, Zipkin
RNF-12	Pruebas automatizadas de integración con base real, de interfaz, de humo y de contrato.	Testcontainers, Vitest

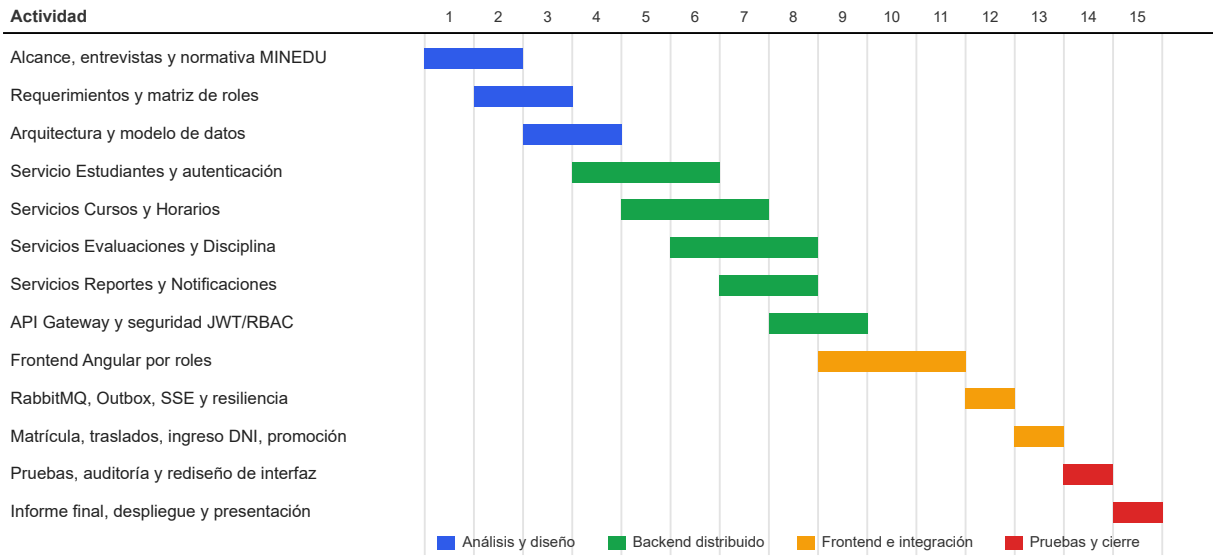
Nota. Elaboración propia.

2.1.3 Plan del Proyecto (Diagrama de Gantt)

El proyecto se desarrolló en quince semanas: análisis y arquitectura (semanas 1 a 3), construcción del backend distribuido (4 a 8), frontend e integración (9 a 11), mensajería, tiempo real y resiliencia (12), funciones del colegio público (13), pruebas y rediseño (14) e informe final, despliegue y presentación (15), como se muestra en la Figura 1.

Figura 1

Plan de desarrollo: diagrama de Gantt (15 semanas)



Nota. Elaboración propia.

2.2 Arquitectura del Sistema

2.2.1 Descripción Jerárquica

La arquitectura se organiza en cinco niveles (ver Figura 2):

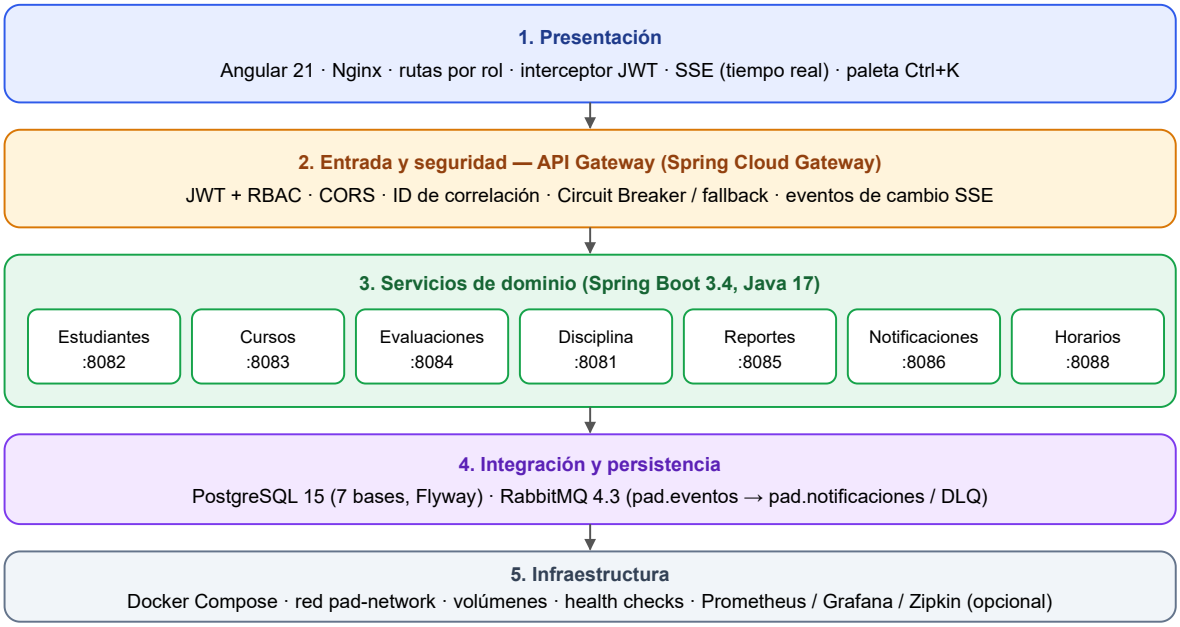
1. **Presentación:** aplicación Angular servida por Nginx, con rutas protegidas por rol, interceptor JWT con reintento automático y conexión SSE para tiempo real.
2. **Entrada y seguridad:** API Gateway que valida el JWT, aplica RBAC, agrega el ID de correlación, protege cada ruta con un *circuit breaker* y publica los eventos de cambio.
3. **Servicios de dominio:** siete microservicios Spring Boot que exponen los casos de uso del colegio.
4. **Integración y persistencia:** RabbitMQ para mensajería asíncrona y PostgreSQL con una base por servicio, versionada con Flyway.

5. **Infraestructura:** contenedores Docker, red interna `pad-network`, volúmenes persistentes, *health checks* y observabilidad opcional.

Dentro de cada microservicio se mantiene la jerarquía Controlador → DTO → Servicio → Entidad/Repositorio → Base de datos.

Figura 2

Descripción jerárquica de la arquitectura



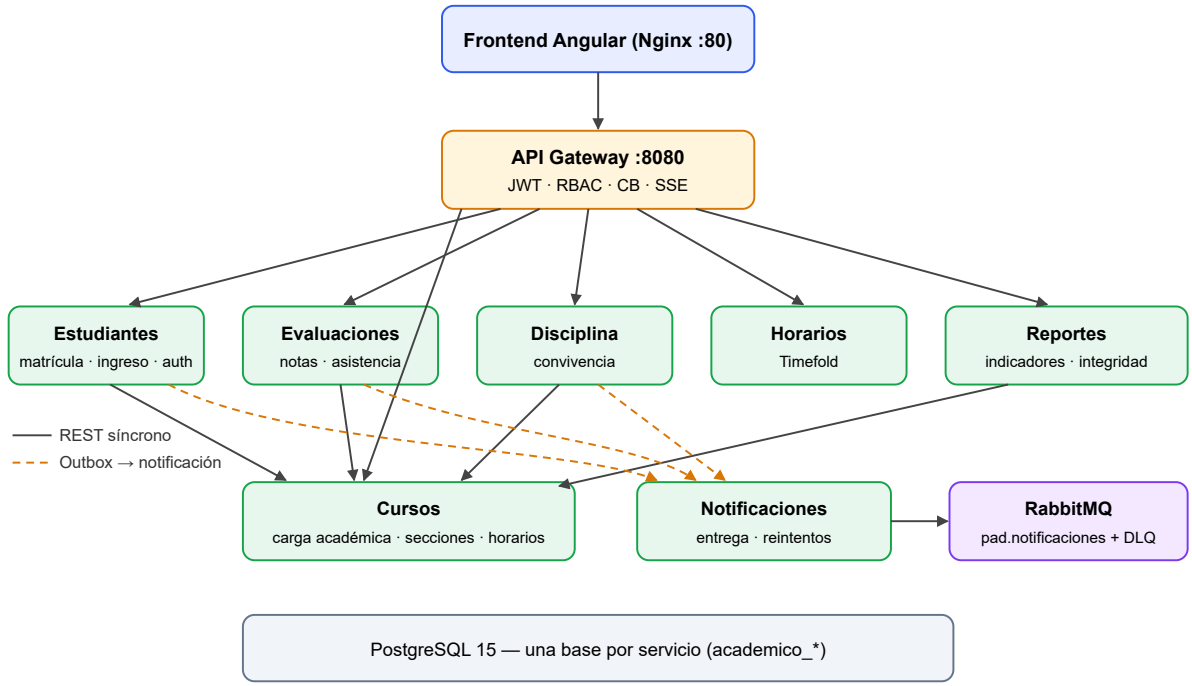
Nota. Elaboración propia.

2.2.2 Diagrama de Módulos

El sistema se divide por capacidades de negocio y no por tablas compartidas. El frontend solo conoce al gateway. Los servicios Estudiantes, Evaluaciones y Disciplina consultan a Cursos la carga académica del docente y registran sus notificaciones en una *outbox* que se entrega al servicio Notificaciones; Reportes consulta a los demás servicios para consolidar indicadores y detectar referencias rotas (ver Figura 3).

Figura 3

Diagrama de módulos y dependencias



Nota. Las flechas continuas representan llamadas REST síncronas protegidas con JWT o con clave interna; las discontinuas, la entrega diferida de notificaciones desde la tabla *outbox* de cada servicio. Elaboración propia.

2.2.3 Descripción Individual de los Módulos

La Tabla 6 describe la responsabilidad, los datos propios y las funcionalidades relevantes de cada módulo.

Tabla 6

Descripción individual de los módulos

Módulo	Datos propios (base)	Funcionalidades relevantes
Frontend Angular	Sesión JWT y preferencias en el navegador	Menú por rol tipo cuaderno, inicio con «Mi lista de hoy», paleta Ctrl+K, «Continuar donde te quedaste», modo pizarra, tiempo real, exportación e impresión.
API Gateway	Sin base (sin estado)	Rutas por prefijo, JWT/RBAC, CORS, *circuit breaker* con *fallback*, reintentos, correlación y difusión SSE de cambios por audiencia de rol.

Módulo	Datos propios (base)	Funcionalidades relevantes
Estudiantes	`academico_estudiantes`: usuarios, estudiantes, apoderados, matrículas, áreas pendientes, ingresos, documentos, consentimientos, fichas médicas, derivaciones, *outbox*	Autenticación, matrícula y trayectoria, traslados, promoción, recuperación, ingreso por DNI, portal del apoderado, auditoría Envers.
Cursos	`academico_cursos`: niveles, grados, secciones, periodos, cursos, docentes, aulas, asignaciones, horarios, tickets	Estructura curricular, carga académica, reparto entre docentes, guardado atómico de horarios por lote con detección de cruces.
Evaluaciones	`academico_evaluaciones`: notas, sesiones y asistencias, estado y auditoría de bimestres, *outbox*	Notas por competencias AD/A/B/C, asistencia por sesión, cierre y reapertura de bimestres.
Disciplina	`academico_disciplina`: reportes disciplinarios, *outbox*	Incidencias de convivencia y aviso al apoderado.
Reportes	`academico_reportes`: actividades, ejecuciones e incidencias de integridad	Indicadores del inicio, alertas de deserción, actividad reciente, reconciliación de referencias entre servicios.
Notificaciones	`academico_notificaciones`: notificaciones	Cola RabbitMQ, proveedores de correo/SMS/WhatsApp, reintentos con espera creciente y DLQ.
Horarios	`academico_horarios`: parámetros, *timeslot*, *room*, *lesson* y réplica académica	Motor de optimización Timefold con restricciones duras y blandas; iniciar, detener y consultar el resultado.

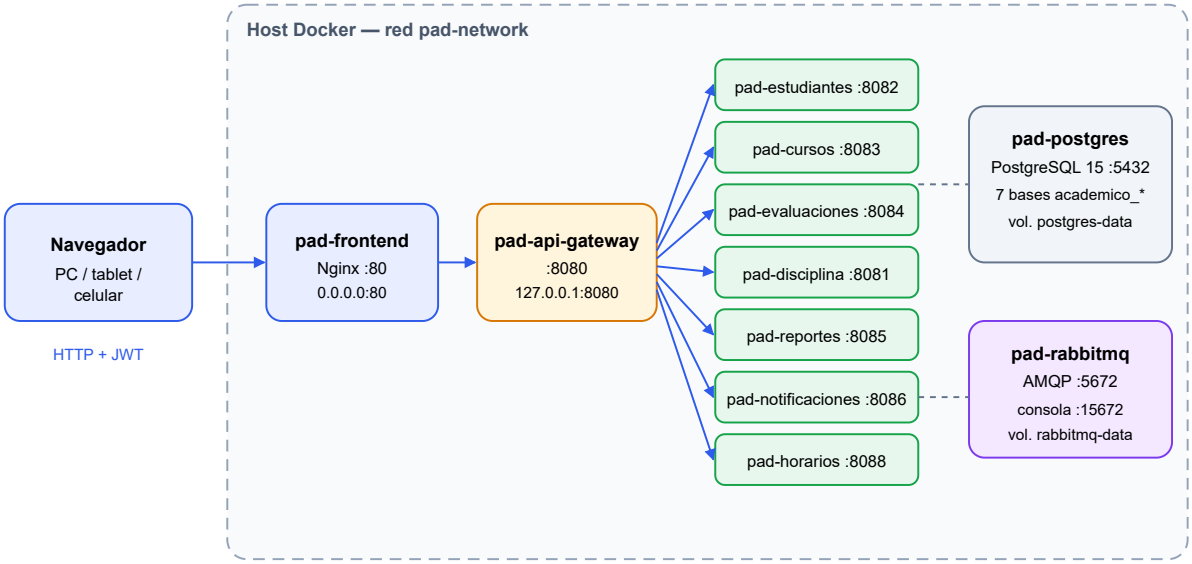
Nota. Elaboración propia a partir del código fuente y de las bases en ejecución.

2.2.4 Arquitectura de los Servicios Web y Topología de Red

A diferencia de la primera versión del proyecto, que no tenía gateway, la implementación final incorpora un **API Gateway** como único punto de entrada. El navegador solo se comunica con Nginx en el puerto 80; Nginx reenvía `/api/**` al gateway, que valida el token y enruta hacia el servicio correspondiente por el nombre del contenedor (DNS interno de Docker). Los servicios de dominio no publican puertos al exterior. La Figura 4 muestra el despliegue lógico.

Figura 4

Diagrama de despliegue en la red Docker `pad-network`

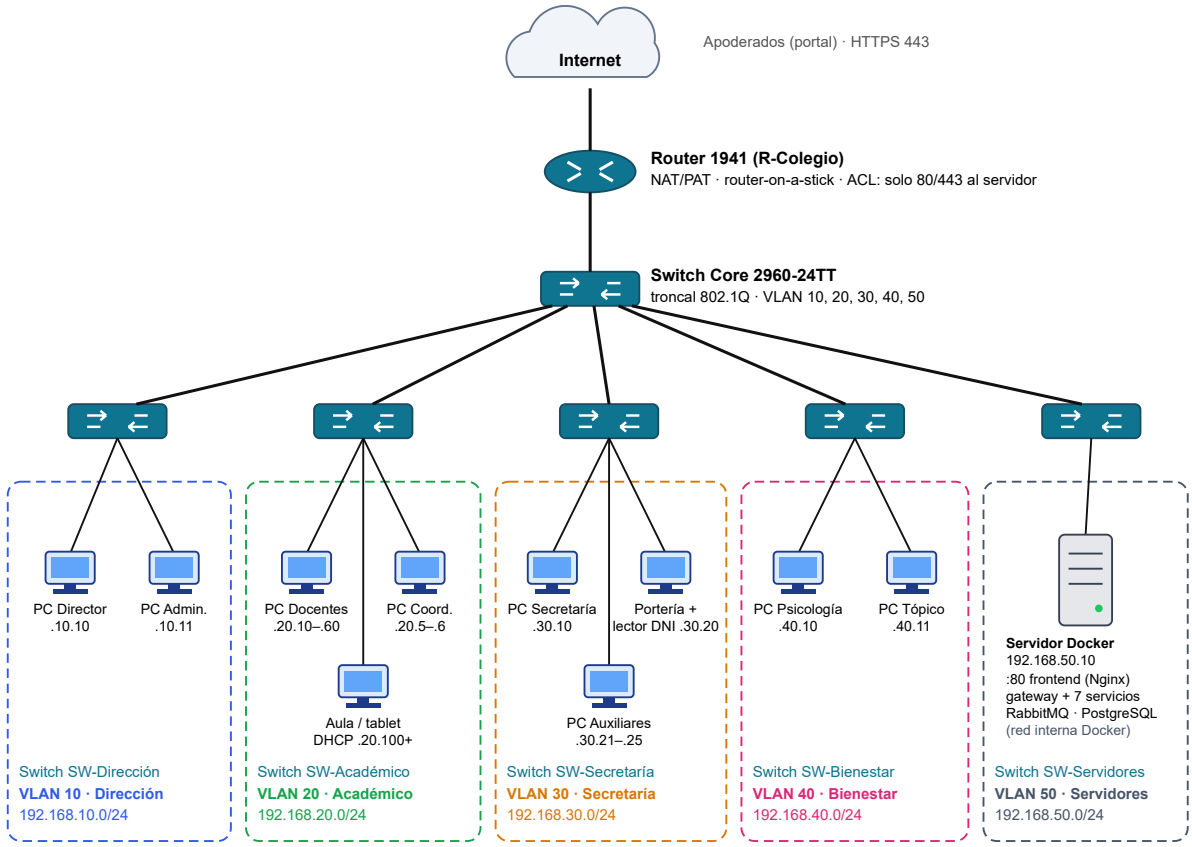


Nota. Las flechas azules representan HTTP con JWT desde el navegador hasta los servicios a través de Nginx y el gateway. Los servicios de dominio solo exponen su puerto dentro de la red Docker. Elaboración propia a partir de `docker-compose.yml`.

La topología física propuesta para el colegio se representa en notación Cisco en la Figura 5. La red se segmenta en cinco VLAN según el área de trabajo: Dirección y Administración (VLAN 10), Académico (VLAN 20: docentes, coordinación y equipos de aula), Secretaría y portería (VLAN 30: incluye el equipo con lector de código de barras para el DNI), Bienestar (VLAN 40: psicología y tópico) y Servidores (VLAN 50). Los switches de acceso 2960-24TT se enlazan por troncales 802.1Q con el switch core, y el router 1941 realiza el enrutamiento entre VLAN (*router-on-a-stick*) y la traducción NAT hacia Internet. De acuerdo con la implementación, existe un único **servidor Docker** que aloja los once contenedores; solo su puerto web (80, o 443 con HTTPS en producción) es accesible desde las VLAN de usuarios y, mediante NAT, para el portal de los apoderados. El gateway, los microservicios, RabbitMQ y PostgreSQL permanecen en la red interna de Docker y no son alcanzables desde los equipos de los usuarios.

Figura 5

Topología física de red de la institución (notación Cisco)



Nota. Diagrama de red coherente con el despliegue de la Figura 4: un servidor Docker (192.168.50.10) con el frontend, el gateway, los siete microservicios, RabbitMQ y PostgreSQL. Modelos de equipos de Cisco Packet Tracer (router 1941 y switches 2960-24TT). Elaboración propia.

La Tabla 7 resume el direccionamiento de la topología y su correspondencia con el sistema.

Tabla 7

Plan de direccionamiento y VLAN

VLAN / red	Equipos	Direcciones	Uso en el sistema
10 · 192.168.10.0/24	PC Director, PC Administración del sistema	.10.10 – .10.11; gateway .10.1	Indicadores, auditoría, usuarios y roles.
20 · 192.168.20.0/24	PC Coordinación, PC docentes, equipos de aula	.20.5 – .20.60 fijas; DHCP .20.100+; gateway .20.1	Carga y horarios, notas (incluida voz), asistencia, convivencia.
30 · 192.168.30.0/24	PC Secretaría, portería con lector de DNI, auxiliares	.30.10 – .30.25; gateway .30.1	Matrícula, traslados, promoción, ingreso al colegio con DNI.

VLAN / red	Equipos	Direcciones	Uso en el sistema
40 · 192.168.40.0/24	PC Psicología, PC Tópico	.40.10 – .40.11; gateway .40.1	Derivaciones, fichas médicas y atenciones (datos sensibles aislados).
50 · 192.168.50.0/24	Servidor Docker	.50.10; gateway .50.1	Contenedores `pad-*`; publica solo el puerto 80/443.
Internet (NAT)	Apoderados desde su celular	IP pública del colegio → .50.10:443	Portal del apoderado (notas, asistencia y comunicaciones).

Nota. En la PC de cada VLAN, la plataforma se abre en el navegador con `http://192.168.50.10`. Elaboración propia.

2.2.5 Descripción del Middleware de Comunicaciones

El middleware está compuesto por los siguientes mecanismos, todos implementados y en ejecución:

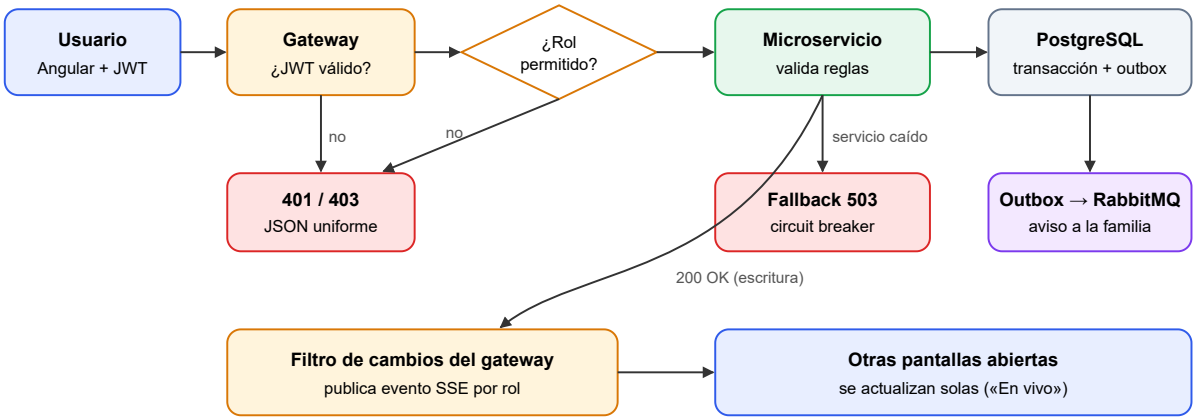
- **Spring Cloud Gateway (WebFlux):** enrutamiento por prefijo (`/api/estudiantes/**`, `/api/notas/**`, `/api/horarios/**`, etc.), validación JWT, RBAC y CORS.
- **Resilience4j:** un *circuit breaker* por ruta con límite de 25 s y respuesta de respaldo (`/fallback/{servicio}`), con tiempos de conexión de 3 s y de respuesta de 30 s. El frontend reintenta automáticamente una vez las lecturas fallidas (0/502/503/504).
- **Server-Sent Events:** un filtro global del gateway detecta cada escritura exitosa (POST, PUT, PATCH o DELETE) y publica un evento de cambio en `/api/tiempo-real`, filtrado por la audiencia de cada rol; las pantallas abiertas recargan solo el módulo afectado.
- **REST interno:** clientes `RestClient`/`WebClient` entre servicios, autenticados con `INTERNAL_API_KEY` y propagando el ID de correlación (`X-Correlation-Id`).
- **Patrón Outbox + RabbitMQ:** Estudiantes, Evaluaciones y Disciplina guardan la notificación en `notification_outbox` en la misma transacción; un proceso programado la entrega a Notificaciones, que la publica en el intercambio `pad.eventos`, la consume desde `pad.notificaciones` y, si falla definitivamente, la deriva a `pad.notificaciones.dlq`.
- **Manejadores globales de errores** que devuelven 4xx para solicitudes inválidas y 5xx solo para fallas reales, siempre en JSON.

- **Observabilidad:** Actuator (*health*), Micrometer/Prometheus y trazas OpenTelemetry hacia Zipkin.

La Figura 6 representa el recorrido de una solicitud desde el navegador hasta la base de datos y la difusión del cambio a los demás usuarios.

Figura 6

Flujo de comunicación (solicitud-respuesta y tiempo real)

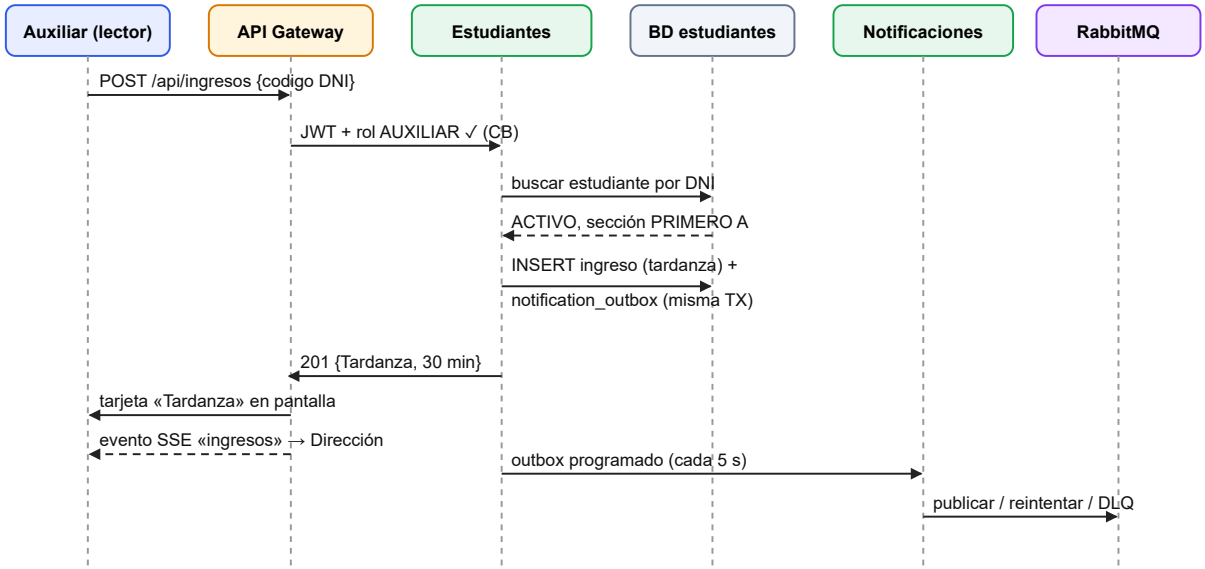


Nota. Elaboración propia.

La Figura 7 muestra la secuencia distribuida del registro de ingreso al colegio con el DNI, una de las operaciones de mayor frecuencia (cientos de lecturas por la mañana).

Figura 7

Secuencia de comunicación distribuida durante el ingreso al colegio con DNI



Nota. La respuesta al portero no espera el envío del mensaje a la familia: la notificación se confirma en la misma transacción y se entrega de forma asíncrona. Si Notificaciones no está disponible, el registro de ingreso no se pierde. Elaboración propia.

2.2.6 Dependencias Externas

Las dependencias externas utilizadas en el proyecto se listan en la Tabla 8.

Tabla 8

Dependencias externas del proyecto

Componente	**Versión / uso en el proyecto**
Java (Eclipse Temurin)	17, lenguaje del gateway y de los siete microservicios.
Spring Boot	3.4.3: Web MVC, Data JPA, Security, Validation, Actuator, AMQP.
Spring Cloud Gateway	2024.0.1 (WebFlux): enrutamiento, filtros globales y SSE.
Resilience4j	*Circuit breaker* y límite de tiempo por ruta.
JJWT	Emisión y validación de tokens JWT HS256.
Hibernate ORM + Envers	Mapeo objeto-relacional y auditoría versionada.
PostgreSQL	15 Alpine; siete bases lógicas.
Flyway	25 migraciones versionadas en total entre los siete servicios.
RabbitMQ	4.3 con consola de administración; colas durables y DLQ.
Timefold Solver	Optimización de horarios con restricciones.
Springdoc OpenAPI	Documentación Swagger de las APIs.
Micrometer, OpenTelemetry, Zipkin, Prometheus, Grafana	Métricas, trazas distribuidas y tableros (perfil `observability`).
Testcontainers + JUnit 5	Pruebas de integración contra PostgreSQL real en contenedor.
Angular	21.2 con componentes *standalone*, *signals* y Zone.js.
TypeScript / RxJS	5.9 / 7.8.
Bootstrap + Bootstrap Icons	5.3: base de estilos e iconografía.
ZXing (@zxing/browser)	0.2.1: lectura del código de barras del DNI con la cámara.
Web Speech API	Dictado de notas por voz en español (es-PE).
Vitest / Playwright	Pruebas unitarias del frontend y pruebas de navegador.
Nginx	Servidor web del frontend y proxy inverso hacia el gateway.
Docker y Docker Compose	Construcción y orquestación de los once contenedores.

Nota. Versiones tomadas de los archivos `pom.xml`, `package.json` y `docker-compose.yml`. Elaboración propia.

2.2.7 Fundamentos de Sistemas Distribuidos Aplicados

Un sistema distribuido es un conjunto de procesos autónomos que se comunican por una red y se presentan al usuario como un sistema único y coherente (Tanenbaum & van Steen, 2017). La plataforma cumple esa definición: once procesos independientes en contenedores colaboran por HTTP y AMQP, y el docente o la secretaria perciben una sola aplicación web. A continuación se explica cómo se resolvieron los problemas propios de esta clase de sistemas.

Transparencia. El usuario no sabe en qué servicio vive cada dato (transparencia de ubicación): el navegador solo conoce `/api/**` y el gateway decide el destino por prefijo de ruta. Los servicios se ubican por nombre DNS dentro de Docker (`servicio-cursos:8083`), por lo que pueden moverse o replicarse sin cambiar el frontend. La transparencia de fallos es parcial y deliberada: si un servicio cae, el usuario recibe un mensaje claro en lugar de un error técnico.

Comunicación síncrona y asíncrona. Se usa comunicación síncrona (solicitud-respuesta REST) cuando el usuario necesita la respuesta para continuar; por ejemplo, Evaluaciones consulta a Cursos si el docente tiene carga en una sección antes de aceptar una nota. Se usa comunicación asíncrona cuando la operación puede completarse después sin afectar al usuario: los avisos a las familias pasan por una tabla *outbox* y por RabbitMQ, desacoplando en el tiempo al emisor del receptor. El tiempo real se resuelve con *Server-Sent Events*, un canal unidireccional del servidor al navegador apropiado para notificar cambios.

Consistencia y teorema CAP. Ante una partición de red, un sistema distribuido debe elegir entre consistencia y disponibilidad (Brewer, 2012). Dentro de cada base de datos se garantiza consistencia fuerte con transacciones ACID. Entre servicios se acepta **consistencia eventual** donde no compromete la operación: una notificación puede llegar segundos después, los indicadores del inicio se recalculan y las referencias lógicas entre bases se verifican periódicamente con la reconciliación de integridad del servicio Reportes. Para operaciones críticas, como registrar una nota, se prefiere la consistencia: si Cursos no puede confirmar la carga del docente, la operación se rechaza en lugar de aceptar un dato dudoso.

Tolerancia a fallos. Se aplicaron varias tácticas complementarias: (a) límites de tiempo (conexión de 3 s, respuesta de 30 s y límite de 25 s por ruta en el gateway), para que una llamada lenta no bloquee hilos indefinidamente; (b) *circuit breaker* por servicio con ventana de 10 llamadas, umbral de fallo del 50 %, 10 s en estado abierto y 3 llamadas de

prueba en estado semiabierto; al abrirse, el gateway responde de inmediato con un 503 controlado; (c) reintento automático de lecturas en el frontend ante errores 0, 502, 503 o 504; (d) reintentos con espera exponencial en la *outbox* (2^n segundos, máximo 300 s) hasta 10 intentos, y (e) cola de mensajes fallidos (DLQ) en RabbitMQ para no perder mensajes envenenados. La prueba de resiliencia del Anexo B5 lo demuestra: al detener Cursos el resto del sistema siguió atendiendo y el servicio se reincorporó solo al reiniciarse.

Atomicidad sin transacciones distribuidas. En lugar de un protocolo de confirmación en dos fases (2PC), costoso y bloqueante, se usó el patrón **Transactional Outbox**: el cambio de negocio y el mensaje pendiente se guardan en la misma transacción local, y un proceso programado cada 5 s publica los pendientes. Así se evita el «doble escritura» inconsistente (guardar el dato pero perder el aviso, o avisar algo que no se guardó).

Idempotencia y entrega al menos una vez. Como un mensaje puede reintentarse, el receptor debe tolerar duplicados. Cada evento lleva un `evento_id` único y la base de Notificaciones impone `UNIQUE (evento_id, canal)`, por lo que un reintento no genera un segundo SMS. Del mismo modo, `UNIQUE (estudiante_id, fecha)` en `ingresos_colegio` impide registrar dos veces el mismo DNI en el día aunque dos lectores lo escaneen simultáneamente.

Tiempo y concurrencia. En un sistema distribuido no se puede confiar en el reloj del cliente. La hora de ingreso y la tardanza se calculan en el servidor con la zona horaria `America/Lima`. Para cambios concurrentes sobre el estado de un bimestre se usa control optimista con un campo `version`: si dos coordinadores lo modifican a la vez, el segundo recibe un conflicto en lugar de sobrescribir.

Escalabilidad y observabilidad. Los servicios no guardan sesión (JWT sin estado), por lo que pueden ejecutarse varias réplicas detrás del gateway. Cada solicitud recibe un `X-Correlation-Id` que se propaga entre servicios y aparece en registros, notificaciones y trazas de Zipkin, lo que permite reconstruir una operación que atravesó varios procesos. La Tabla 9 resume estas propiedades.

Tabla 9

Problemas de sistemas distribuidos y mecanismos implementados

Propiedad	Problema que resuelve	Mecanismo en el sistema	Evidencia
Transparencia	El usuario no debe conocer la distribución.	Gateway único, enrutamiento por prefijo y DNS interno de Docker.	Fig. 4
Disponibilidad	La caída de un servicio no debe detener al colegio.	*Circuit breaker*, *fallback* 503, *health checks* y reinicio de contenedores.	Anexo B5
Latencia acotada	Un servicio lento bloquea a los demás.	Timeouts de 3 s / 30 s y límite de 25 s por ruta.	`application.yml`
Entrega fiable	Los mensajes pueden perderse si el receptor falla.	Outbox transaccional, reintentos exponenciales, colas durables y DLQ.	Fig. 26; Fig. A4
Idempotencia	Los reintentos generan duplicados.	`evento_id` único; `UNIQUE (evento_id, canal)` y `(estudiante_id, fecha)`.	Tabla 18
Consistencia	Datos repartidos en siete bases.	ACID local, consistencia eventual entre servicios y reconciliación de integridad.	3.1.6
Concurrencia	Escrituras simultáneas sobre el mismo recurso.	Control optimista (`version`) y restricciones únicas.	Contrato
Coordinación del tiempo	Relojes de clientes distintos.	Hora del servidor en zona `America/Lima`.	Fig. 19
Observabilidad	Difícil seguir una operación entre procesos.	ID de correlación, Micrometer, OpenTelemetry y Zipkin.	Perfil `observability`
Escalabilidad	Picos de uso (ingreso de 7:30 a 8:00).	Servicios sin estado y escalado horizontal por réplicas.	RNF-02

Nota. Elaboración propia.

2.2.8 Decisiones de Arquitectura de Software

La arquitectura de software es el conjunto de estructuras necesarias para razonar sobre un sistema, formadas por sus elementos, las relaciones entre ellos y sus propiedades (Bass et al., 2021). Esta sección justifica el diseño desde esa disciplina.

Estilo arquitectónico. Se eligió el estilo de **microservicios** (Newman, 2021), con una **arquitectura en capas** dentro de cada servicio. Los límites de los servicios se trazaron siguiendo los **contextos delimitados** del dominio escolar (Evans, 2003): identidad y vida del estudiante, organización académica, evaluación, convivencia, comunicaciones, analítica y

planificación de horarios. Cada contexto tiene su propio modelo y vocabulario; por ejemplo, «sección» es una entidad completa en Cursos, pero en Evaluaciones es solo un identificador de referencia.

Vistas del modelo 4+1. La arquitectura se documenta con las cinco vistas de Kruchten (1995): la *vista lógica* con el diagrama de módulos (Figura 3) y el modelo de datos (Figura 10); la *vista de procesos* con el flujo de solicitud y la secuencia del ingreso (Figuras 6 y 7); la *vista de desarrollo* con la estructura del repositorio (apartado 3.2.2); la *vista física* con el despliegue en Docker y la topología de red (Figuras 4 y 5), y la *vista de escenarios* con los casos de uso y el proceso del año escolar (Figuras 8 y 9).

Patrones aplicados. La Tabla 10 lista los patrones de arquitectura y de diseño utilizados y el problema que resuelve cada uno.

Tabla 10

Patrones de arquitectura y diseño aplicados

Patrón	Problema	Aplicación en el sistema
API Gateway	Muchos servicios expuestos al cliente, con seguridad repetida.	Punto de entrada único con JWT, RBAC, CORS, resiliencia y SSE.
Database per Service	Acoplamiento por base compartida.	Siete bases; ningún servicio lee tablas ajenas.
Transactional Outbox	Guardar un dato y publicar un evento de forma atómica.	`notification_outbox` en Estudiantes, Evaluaciones y Disciplina.
Circuit Breaker	Fallos en cascada por un servicio caído.	Resilience4j por ruta con respuesta de respaldo.
Publicar/Suscribir	Varios clientes deben enterarse de un cambio.	Eventos SSE por audiencia de rol; intercambio `pad.eventos` en RabbitMQ.
Dead Letter Queue	Mensajes que nunca podrán procesarse.	`pad.notificaciones.dlq`.
Correlation ID	Rastrear una operación entre procesos.	Filtro del gateway y propagación en REST y mensajes.
Capas (Layered)	Mezcla de responsabilidades.	Controlador → Servicio → Repositorio → Entidad.
DTO	Exponer entidades internas acopla API y base.	Objetos de entrada/salida validados en cada endpoint.
Repository	Acceso a datos disperso.	Interfaces Spring Data JPA por agregado.

Patrón	Problema	Aplicación en el sistema
Idempotent Receiver	Duplicados por reintentos.	Claves únicas de evento y *upsert* por clave natural en notas.

Nota. Elaboración propia con base en Richardson (2018) y Newman (2021).

Atributos de calidad. Siguiendo el modelo ISO/IEC 25010, los atributos priorizados para un colegio público fueron la seguridad (datos de menores de edad), la disponibilidad en la hora de ingreso, la usabilidad para personal sin formación técnica y la mantenibilidad para un equipo pequeño. La Tabla 11 relaciona cada atributo con las tácticas aplicadas (Bass et al., 2021).

Tabla 11

Atributos de calidad y tácticas arquitectónicas

Atributo	Escenario	Tácticas aplicadas
Seguridad	Un docente intenta ver notas de otra sección.	Autenticación JWT, autorización RBAC en gateway y servicios, alcance por carga académica, mínimo privilegio, BCrypt, auditoría.
Disponibilidad	Cursos se cae a las 7:45 durante el ingreso.	Aislamiento por servicio, *circuit breaker*, *health checks*, reinicio automático; Portería depende solo de Estudiantes.
Rendimiento	500 estudiantes escanean su DNI en 30 minutos.	Índices, consultas paginadas, pool de conexiones, envío de avisos asíncrono.
Usabilidad	Una auxiliar nueva debe registrar el ingreso sin capacitación.	Menú por rol, «Mi lista de hoy», Ctrl + K, flujo grado → sección, lector USB o cámara.
Mantenibilidad	Agregar el módulo de recuperación pedagógica.	Alta cohesión por contexto, bajo acoplamiento, migraciones Flyway, pruebas automatizadas.
Portabilidad	Llevar el sistema a un servidor de la UGEL.	Contenedores, configuración por variables de entorno, un solo comando de despliegue.
Interoperabilidad	Integrar SMS o WhatsApp.	Contratos REST JSON documentados con OpenAPI; proveedores configurables.

Nota. Elaboración propia.

Registro de decisiones de arquitectura. La Tabla 12 documenta las decisiones principales, las alternativas evaluadas y sus consecuencias, al estilo de los *Architecture Decision Records*.

Tabla 12

Decisiones de arquitectura, alternativas y consecuencias

ID	Decisión	Alternativas	Justificación y consecuencias
AD-01	Microservicios por contexto del dominio	Monolito modular	Aísla fallos y permite evolucionar cada área; a cambio, exige gateway, mensajería y pruebas de integración.
AD-02	Una instancia PostgreSQL con siete bases separadas	Siete instancias; esquema compartido	Mantiene la propiedad de los datos con menor consumo de memoria en un servidor modesto; la instancia es un punto único de falla que se mitiga con respaldos cifrados.
AD-03	API Gateway como único punto de entrada	Frontend llamando a cada servicio	Centraliza seguridad, CORS y resiliencia; el gateway se vuelve crítico, pero no guarda estado y puede replicarse.
AD-04	Outbox + RabbitMQ para avisos	Llamada REST directa	El usuario no espera al proveedor de SMS y no se pierden avisos; se acepta consistencia eventual.
AD-05	SSE para tiempo real	WebSocket; consultas periódicas	Basta un canal servidor → cliente, funciona sobre HTTP y atraviesa proxies; menos carga que consultar cada pocos segundos.
AD-06	JWT sin estado	Sesiones en servidor	Permite escalar réplicas sin sesión compartida; la revocación inmediata requiere tokens de corta duración.
AD-07	Referencias lógicas entre bases + reconciliación	Claves foráneas entre bases (imposible); replicación de tablas	Respeta la independencia de los servicios; las referencias rotas se detectan y reportan en lugar de impedirse.
AD-08	Escala literal AD/A/B/C como dato principal	Nota vigesimal	Cumple el CNEB; se conserva un valor numérico opcional para reportes.

Nota. Elaboración propia.

3. Diseño e Implementación

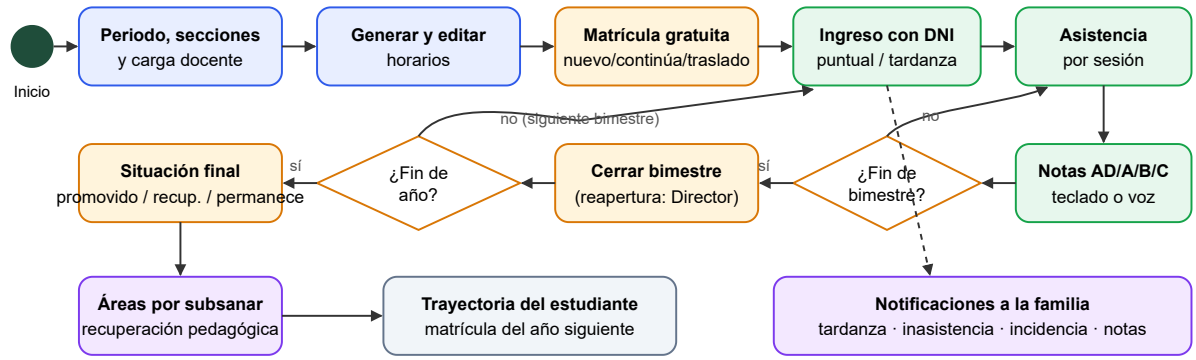
3.1 Diseño del Modelo de Datos

3.1.1 Modelado de Proceso

El proceso principal sigue el año escolar de un colegio público (ver Figura 8). Coordinación académica configura el periodo, las secciones y la carga docente y genera los horarios. Secretaría matricula a los estudiantes (nuevo ingreso, continuidad o traslado). Cada día, el auxiliar registra el ingreso con el DNI y los docentes toman asistencia y registran notas por competencias. Al terminar cada bimestre, Coordinación lo cierra. Al final del año, Secretaría registra la situación final: promovido (con o sin áreas por subsanar), recuperación pedagógica, permanencia o egreso; las áreas pendientes se resuelven en la recuperación y quedan en la trayectoria del estudiante.

Figura 8

Proceso académico del año escolar



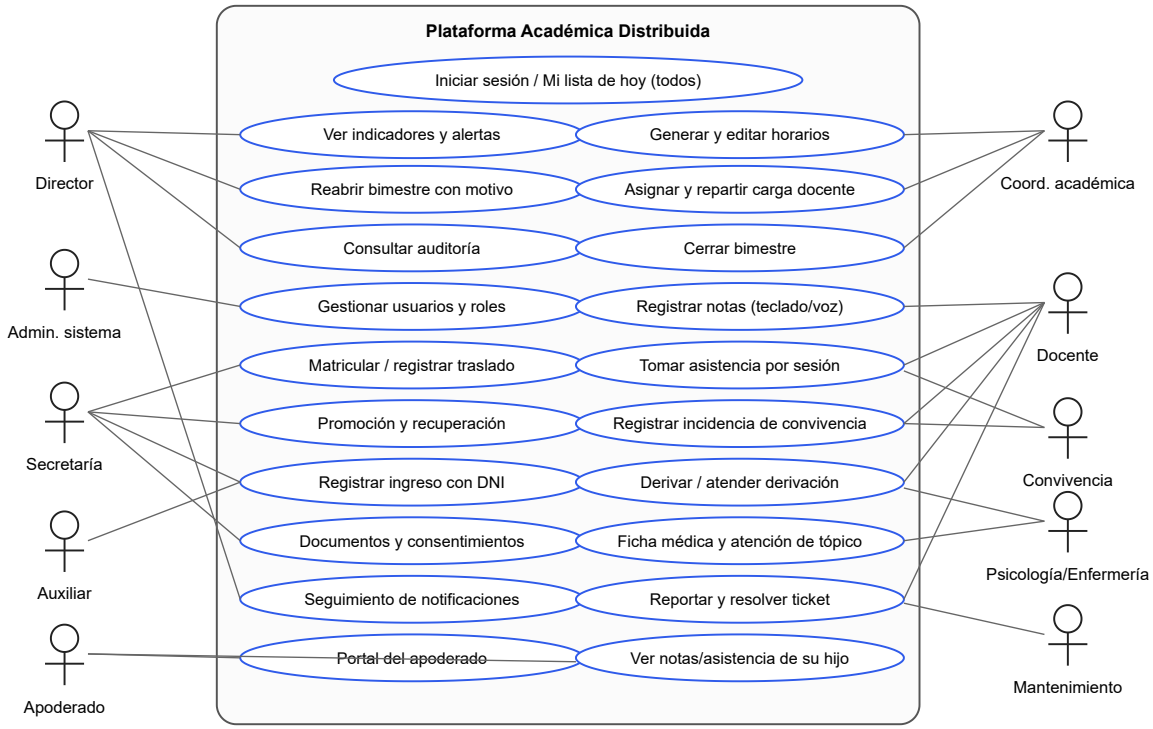
Nota. Elaboración propia.

3.1.2 Diagrama de Casos de Uso

Los casos de uso se distribuyen entre los roles institucionales. La autenticación y el inicio con «Mi lista de hoy» son comunes; las operaciones sensibles aplican además la validación de rol y, para el docente, la de su carga académica (ver Figura 9).

Figura 9

Casos de uso por actor



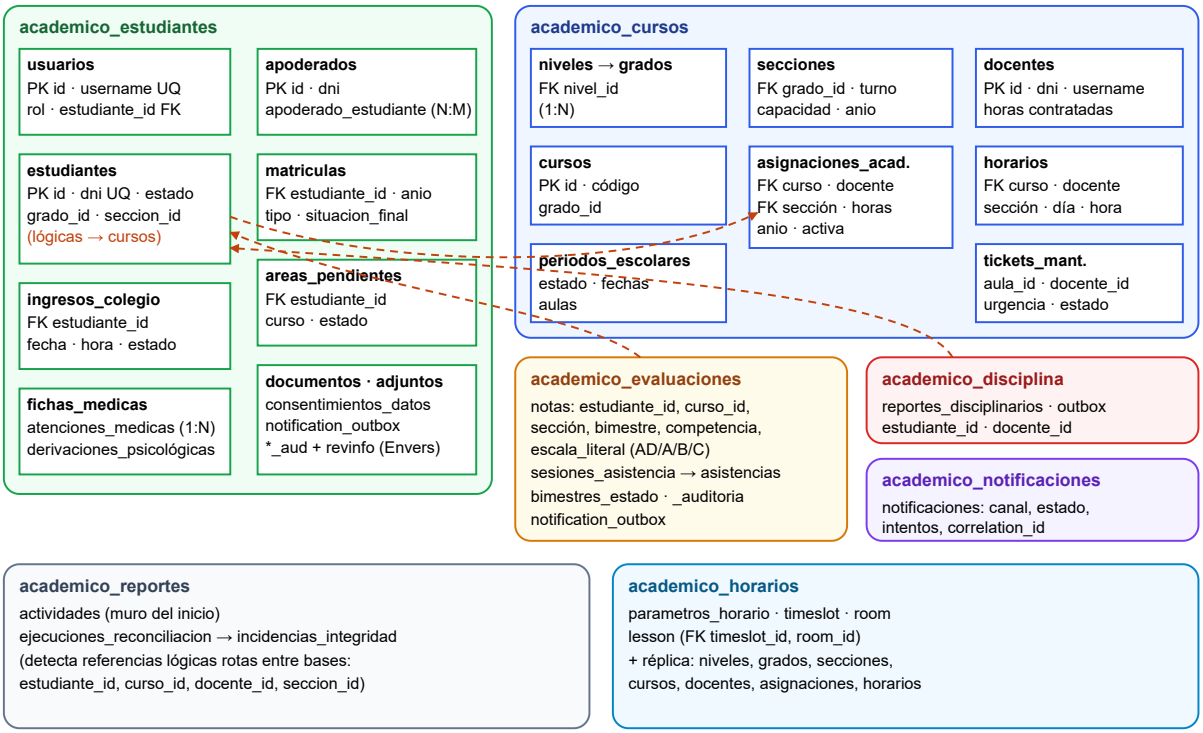
Nota. Elaboración propia.

3.1.3 Diagrama de Entidad-Relación

El modelo contiene 55 tablas distribuidas en siete bases (sin contar las tablas de historial de Flyway; incluye tablas de auditoría Envers y de outbox): `academico_estudiantes` (19), `academico_cursos` (10), `academico_evaluaciones` (6), `academico_disciplina` (2), `academico_reportes` (3), `academico_notificaciones` (1) y `academico_horarios` (14, de las cuales 4 son propias del motor de horarios y 10 una réplica del esquema académico). Las relaciones dentro de una misma base tienen claves foráneas físicas; las relaciones entre bases son referencias lógicas (*soft foreign keys*) que se validan por API y que el servicio Reportes revisa periódicamente mediante la reconciliación de integridad (ver Figura 10).

Figura 10

Modelo entidad-relación consolidado y referencias lógicas entre bases



Nota. Las líneas discontinuas naranjas son referencias lógicas entre bases (*soft foreign keys*); dentro de cada recuadro las relaciones son claves foráneas físicas. Diagrama elaborado a partir de `information_schema` de las bases en ejecución. Elaboración propia.

3.1.4 Descripción de Tablas, Llaves y Relaciones

Las Tablas 13 a 16 describen las tablas principales de cada base de datos.

Tabla 13

Base `academico_estudiantes` (tablas principales)

Tabla	Llaves y relaciones	Descripción
usuarios / usuario_roles	PK id; `username` único; FK opcional a estudiante o apoderado; roles N por usuario	Credenciales BCrypt, rol principal, estado, cambio obligatorio de clave y último acceso.
estudiantes	PK id; DNI único; `grado_id`, `seccion_id` lógicos (Cursos)	Datos personales, estado (ACTIVO, TRASLADADO, EGRESADO…).
apoderados / apoderado_estudiante	PK id; tabla puente con parentesco y permiso de retiro	Familia vinculada; base del portal del apoderado.
matriculas	FK `estudiante_id`; una por año escolar	Tipo (nuevo, continúa, traslado), IE de procedencia/destino con código modular, situación final.

Tabla	Llaves y relaciones	Descripción
areas_pendientes	FK `estudiante_id`	Áreas por subsanar o en recuperación, grado y año de origen, resultado.
ingresos_colegio	FK `estudiante_id`; índice por fecha	Hora de llegada, turno, estado (puntual/tardanza), método (lector/cámara/manual).
fichas_medicas / atenciones_medicas	1:N por FK `ficha_medica_id`	Alergias, seguro, contacto de emergencia y atenciones de tópico.
derivaciones_psicologicas	`estudiante_id`, `docente_derivador_id`	Motivo, prioridad, estado y notas reservadas del psicólogo.
documentos, documentos_adjuntos, consentimientos_datos	FK `estudiante_id` / `apoderado_id`	Legajo digital y decisiones de tratamiento de datos con versión de política y evidencia.
notification_outbox	PK id; `evento_id` único	Mensajes pendientes, intentos, próximo intento y error (patrón Outbox).

Nota. Las tablas `*_aud` y `revinfo` guardan el historial de Hibernate Envers. Elaboración propia.

Tabla 14

Base `academico_cursos`

Tabla	Llaves y relaciones	Descripción
niveles / grados	FK `grados.nivel_id`	Primaria/Secundaria y grados de primero a quinto.
secciones	FK `grado_id`; tutor lógico	Sección, turno (mañana/tarde), capacidad y año escolar.
cursos	PK id; código único	Áreas curriculares del CNEB por grado.
docentes	PK id; DNI y `username` únicos	Especialidad, estado y horas semanales contratadas.
asignaciones_academicas	FK curso, docente y sección	Carga académica (horas por semana); permite varios docentes en un mismo curso.
horarios	FK curso y docente; sección y aula	Bloques por día y hora; se guardan por lote en una sola transacción.
periodos_escolares, aulas, tickets_mantenimiento	PK id; ticket con aula y docente	Calendario escolar, ambientes y solicitudes de mantenimiento.

Nota. Elaboración propia.

Tabla 15

Bases `academico_evaluaciones` y `academico_disciplina`

Tabla	Llaves y relaciones	Descripción
notas	Estudiante, curso, sección, docente (lógicas); índices por estudiante, curso y docente; *upsert* por clave natural	Calificación literal AD/A/B/C (y valor numérico opcional) con observaciones.
sesiones_asistencia / asistencias	Sesión por horario y fecha; registro por estudiante	Apertura y cierre de la sesión, estado (presente, tarde, falta, justificada).
bimestres_estado / bimestres_auditoria	Único por año + bimestre; campo `version`	Estado abierto/cerrado con control optimista y registro de cada cierre o reapertura con motivo.
reportes_disciplinarios	Estudiante, docente y sección (lógicas)	Tipo de incidencia, descripción, estado y notificación al apoderado.

Nota. Ambas bases incluyen su propia tabla `notification_outbox`. Elaboración propia.

Tabla 16

Bases `academico_notificaciones`, `academico_reportes` y `academico_horarios`

Tabla	Llaves y relaciones	Descripción
notificaciones	PK id; `evento_id` único (idempotencia)	Canal, destinatario, estado, intentos, máximo de intentos, error y respuesta del proveedor.
actividades	PK id	Muro de actividad del inicio (periódico mural).
ejecuciones_reconciliacion / incidencias_integridad	FK `ultima_ejecucion`; PK `fingerprint`	Resultado de cada revisión de integridad entre bases y referencias rotas detectadas.
parametros_horario, timeslot, room, lesson	FK `lesson.timeslot_id` y `room_id`	Parámetros y entidades de planificación del motor Timefold.

Nota. Elaboración propia.

3.1.5 Conexión de BD

Cada servicio configura su conexión mediante variables de entorno y usa el nombre del contenedor `postgres` como DNS interno. Las bases se crean al iniciar PostgreSQL mediante el script `backend/init-postgres-dbs.sh`, según se muestra en la Tabla 17.

Tabla 17

Conexión de cada servicio con su base de datos

Servicio	Base de datos	Host interno	Puerto	Migraciones Flyway
Estudiantes	academico_estudiantes	postgres	5432	V1 a V8
Cursos	academico_cursos	postgres	5432	V1 a V3
Evaluaciones	academico_evaluaciones	postgres	5432	V1 a V4
Disciplina	academico_disciplina	postgres	5432	V1 a V3
Reportes	academico_reportes	postgres	5432	V1 a V2
Notificaciones	academico_notificaciones	postgres	5432	V1
Horarios	academico_horarios	postgres	5432	V1 a V4

Nota. Para administración local, PostgreSQL se publica solo en `127.0.0.1:5433`. Elaboración propia.

A continuación se presenta un ejemplo de configuración en `docker-compose.yml`:

```
servicio-estudiantes:
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/academico_estudiantes
    SPRING_DATASOURCE_USERNAME: ${DB_USERNAME}
    SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    SPRING_JPA_HIBERNATE_DDL_AUTO: validate
    JWT_SECRET: ${JWT_SECRET}
    INTERNAL_API_KEY: ${INTERNAL_API_KEY}
  depends_on:
    postgres: { condition: service_healthy }
```

La propiedad `ddl-auto=validate` impide cambios automáticos no controlados: Flyway crea y versiona el esquema y Hibernate solo comprueba que coincida con las entidades. Las contraseñas y secretos se leen del archivo `.env`, que no se versiona; el repositorio solo incluye `.env.example`.

3.1.6 Gestión de la Base de Datos

Esta sección fundamenta el diseño de datos desde la disciplina de gestión de bases de datos (Elmasri & Navathe, 2016; Silberschatz et al., 2020). Las cifras se obtuvieron consultando el catálogo de PostgreSQL (`pg_constraint`, `pg_indexes`) de las bases en ejecución.

Modelo relacional y normalización. Las tablas están en tercera forma normal: cada atributo depende solo de la clave primaria. Por ejemplo, los datos del apoderado no se repiten en cada estudiante; la relación muchos a muchos se resuelve con la tabla puente `apoderado_estudiante`, que además guarda atributos propios de la relación (parentesco y

permiso de retiro). La matrícula se separa del estudiante (`matriculas`, una fila por año) para conservar la trayectoria histórica en lugar de sobrescribirla. Se aceptaron dos desnormalizaciones controladas: `areas_pendientes.curso_nombre` guarda el nombre del área tal como era en el año de origen (una «fotografía» histórica que no debe cambiar si luego se renombra el curso) y la base de Horarios mantiene una réplica de lectura del catálogo académico para que el motor de optimización trabaje sin consultar a otro servicio.

Integridad. Las reglas se declaran en la propia base, no solo en el código, para que ningún error de programación pueda violarlas. La Tabla 18 resume las restricciones existentes.

Tabla 18

Restricciones de integridad e índices por base de datos

Base	PK	FK	UNIQUE	CHECK	Índices	Tamaño
academico_estudiantes	19	15	9	8	41	9,5 MB
academico_cursos	10	7	5	3	23	8,6 MB
academico_evaluaciones	6	0	3	2	21	8,7 MB
academico_disciplina	2	0	1	0	9	7,6 MB
academico_reportes	3	1	0	0	6	9,5 MB
academico_notificaciones	1	0	1	2	6	7,5 MB
academico_horarios	14	9	5	4	27	8,0 MB
Total	**55**	**32**	**24**	**19**	**133**	

Nota. Sin contar la tabla `flyway_schema_history` de cada base. Los índices incluyen los creados por claves primarias y restricciones únicas. Datos obtenidos de `pg_constraint` y `pg_indexes` el 13/10/2026. Elaboración propia.

- **Integridad de entidad:** las 55 tablas tienen clave primaria.
- **Integridad referencial:** 32 claves foráneas dentro de cada base, por ejemplo `matriculas.estudiante_id → estudiantes` o `asignaciones_academicas.docente_id → docentes`. Entre bases se usan referencias lógicas validadas por API y por la reconciliación de integridad.
- **Integridad de dominio:** 19 restricciones CHECK limitan los valores válidos, por ejemplo `ck_matricula_tipo` (NUEVO_INGRESO, CONTINUIDAD, TRASLADO_ENTRANTE), `ck_area_calificacion` (AD, A, B, C), `ck_ingreso_estado` (PUNTUAL, TARDANZA) o `horas_semanales BETWEEN 1 AND 40`.

- **Reglas de negocio como restricciones:** 24 restricciones UNIQUE, como `uk_matricula_estudiante_anio` (una matrícula por año), `uk_ingreso_estudiante_fecha` (un ingreso por día), `uk_bimestre_anio` o `uk_asignacion_curso_docente_seccion_anio`. En notas, la unicidad por estudiante, curso, sección, bimestre y competencia se garantiza en el servicio con una operación *upsert* por clave natural.

Índices y rendimiento. Se crearon índices sobre las columnas usadas en filtros y uniones frecuentes: `idx_nota_estudiante`, `idx_nota_curso`, `idx_asistencia_fecha`, `idx_sesion_seccion_fecha` o el índice de mensajes pendientes de la *outbox*. También se usan índices parciales, como `uk_usuario_apoderado ... WHERE apoderado_id IS NOT NULL`, que garantizan unicidad solo cuando el valor existe. Los listados se paginan en la base (`LIMIT/OFFSET` mediante `Pageable`) para no transferir miles de filas.

Transacciones y aislamiento. Las operaciones de negocio se delimitan con `@Transactional` (69 métodos en los servicios) y cumplen las propiedades ACID. Se usa el nivel de aislamiento *READ COMMITTED* de PostgreSQL, suficiente porque las situaciones de carrera relevantes se controlan con restricciones únicas y con control optimista. Un ejemplo de atomicidad es el guardado de horarios: todas las secciones de un turno se reemplazan en una sola transacción; si un bloque genera un cruce de docente, se revierte el lote completo y la base nunca queda con un horario a medias.

Control de concurrencia. Se aplica control optimista mediante el campo `version` (anotación `@Version` de JPA) en `bimestres_estado`: la actualización incluye la versión leída y falla si otro usuario la cambió antes. Para las inserciones concurrentes (dos lectores de DNI a la vez, dos reintentos del mismo evento) la protección la dan las restricciones UNIQUE, que la base evalúa de forma atómica.

Evolución del esquema. El esquema se versiona con 25 migraciones Flyway (`V1__...sql` a `V8__...sql`) que se aplican en orden al iniciar cada servicio y quedan registradas en `flyway_schema_history` con su suma de verificación. Hibernate trabaja con `ddl-auto=validate`: nunca altera el esquema, solo comprueba que coincida con las entidades. Así, el mismo esquema se reproduce exactamente en desarrollo, en pruebas (Testcontainers aplica las mismas migraciones) y en producción.

Auditoría. Hibernate Envers registra cada cambio de estudiantes, apoderados y documentos en las tablas `estudiantes_aud`, `apoderados_aud` y `documentos_aud`, con el

número de revisión, la fecha, el actor y el ID de correlación (`revinfo`). Los bimestres llevan su propia bitácora (`bimestres_auditoria`) con el motivo de cada cierre o reapertura.

Seguridad y privacidad de los datos. Las credenciales de la base se inyectan por variables de entorno y no se versionan; PostgreSQL solo escucha en `127.0.0.1` fuera de Docker. Las contraseñas de usuarios se guardan con BCrypt. Los datos sensibles de salud y bienestar están en tablas a las que solo acceden Enfermería y Psicología, y las autorizaciones de tratamiento de datos se registran en `consentimientos_datos` con versión de la política y evidencia, conforme a la Ley N.° 29733 de Protección de Datos Personales.

Respaldo y recuperación. El servicio `backup-postgres` (perfil `operations`) ejecuta `pg_dump` en formato personalizado de cada base, cifra el archivo con AES-256 (OpenSSL con PBKDF2 de 200 000 iteraciones), lo repite cada 24 horas y conserva 30 días de copias (configurable entre 7 y 365). El script `verify-restore.sh` restaura la última copia en una base temporal para comprobar que realmente se puede recuperar, y `ops/tests/backup-restore.ps1` automatiza esa verificación. La persistencia ante reinicios se comprobó en el Anexo B6.

Conexiones. Cada servicio usa un *pool* HikariCP (hasta 15 conexiones en Cursos y Horarios, 10 por defecto en los demás); la suma (≈ 80) queda por debajo del límite `max_connections = 100` de la instancia, dejando margen para el respaldo y la administración.

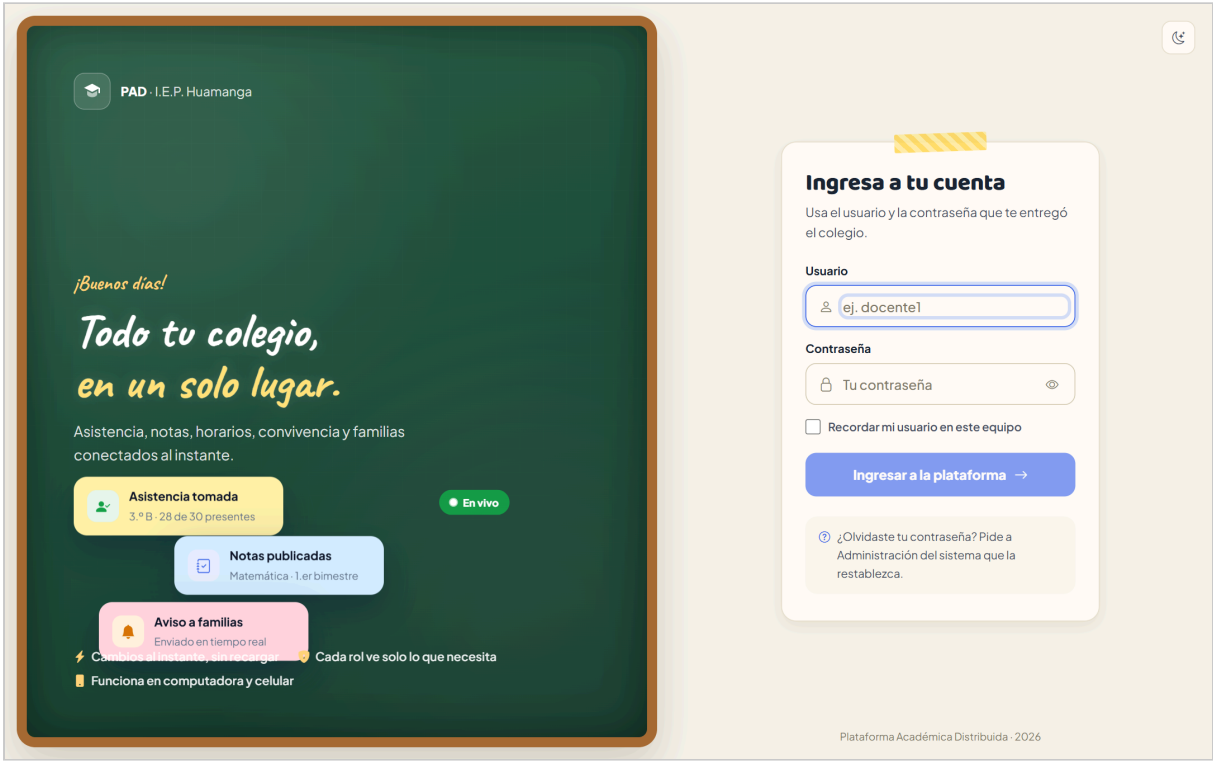
3.2 Implementación

3.2.1 Modelo de Interfaces

La interfaz se diseñó para que cualquier persona del colegio la entienda sin capacitación: tiene temática escolar (fondo de cuaderno cuadriculado, pizarra, notas adhesivas y periódico mural), un menú lateral tipo cuaderno con separadores por área (azul: Académico; verde: Convivencia y bienestar; naranja: Secretaría; morado: Gestión) y una barra superior con buscador universal (Ctrl + K), indicador «En vivo», notificaciones y cambio a modo pizarra (oscuro). Después del login, el sistema lee los roles del JWT y muestra solo los módulos permitidos. Las figuras siguientes son capturas reales del sistema en ejecución, tomadas el 13 de octubre de 2026 con datos de demostración.

Figura 11

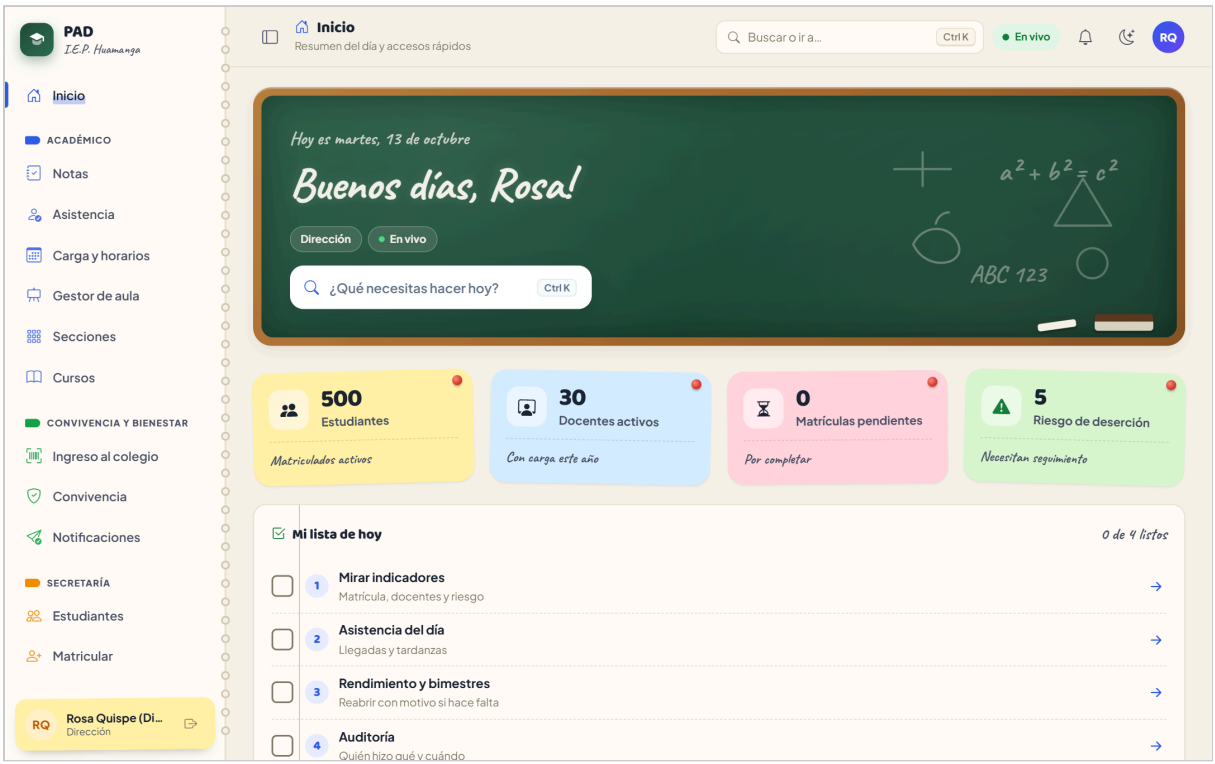
Inicio de sesión con temática de pizarra



Nota. Pantalla de acceso; las cuentas nuevas deben cambiar su contraseña en el primer ingreso. Captura del sistema en ejecución (`http://localhost/login`).

Figura 12

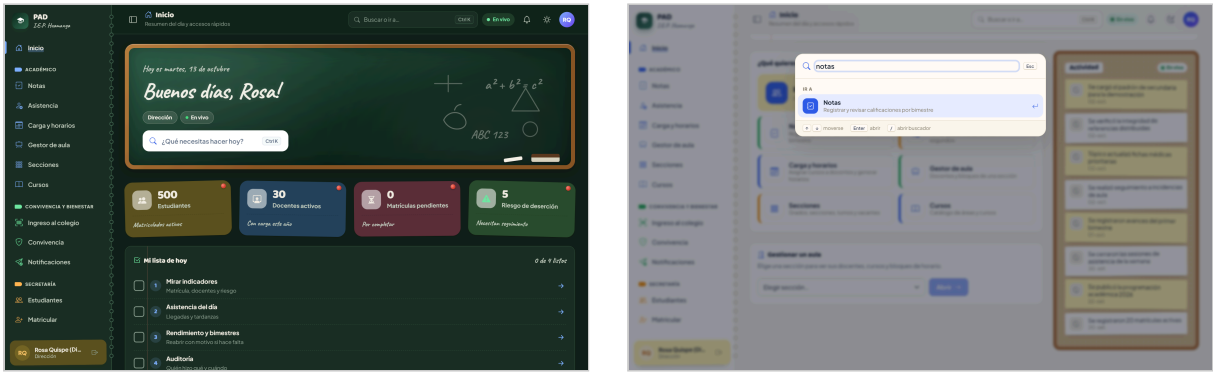
Inicio de la Dirección: pizarra de bienvenida, indicadores y «Mi lista de hoy»



Nota. Los indicadores (500 estudiantes, 30 docentes, 5 en riesgo de deserción) provienen del servicio Reportes; la lista de pasos depende del rol y se reinicia cada día. Captura del sistema en ejecución.

Figura 13

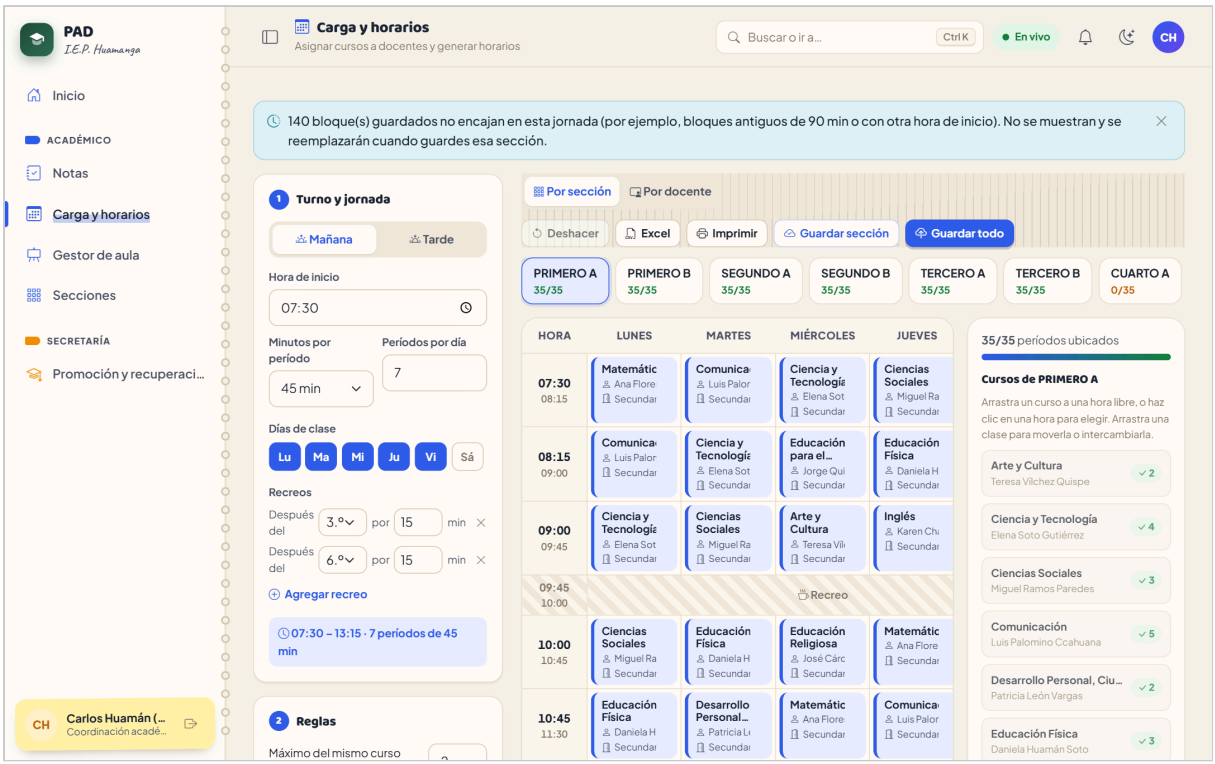
Modo pizarra (tema oscuro) y paleta de comandos Ctrl + K



Nota. A la izquierda, el inicio en modo pizarra; a la derecha, el buscador universal: al escribir «notas» lleva directamente al módulo. Detrás se ve el periódico mural de actividad. Capturas del sistema en ejecución.

Figura 14

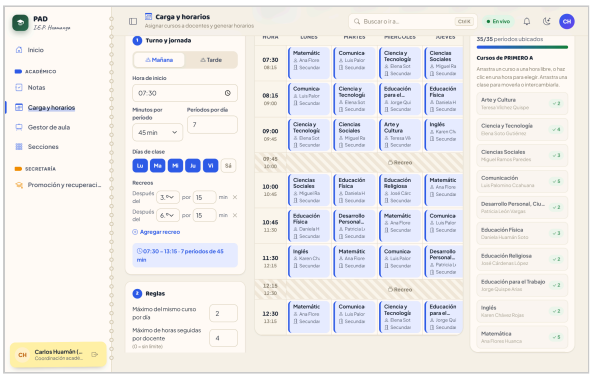
Generador de horarios con parámetros editables (Coordinación académica)

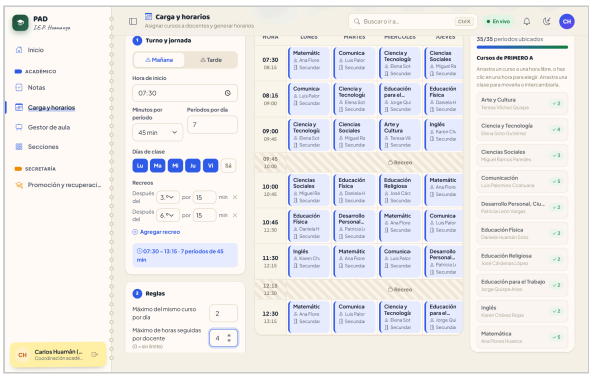


Nota. El coordinador elige turno, hora de inicio, minutos por periodo, periodos por día, días y recreos; el sistema genera el horario de cada sección (35/35 periodos ubicados) y permite arrastrar bloques para editarlo, deshacer, exportar a Excel, imprimir y guardar todas las secciones en una sola transacción. Captura del sistema en ejecución.

Figura 15

Reglas del generador y vista del horario por docente

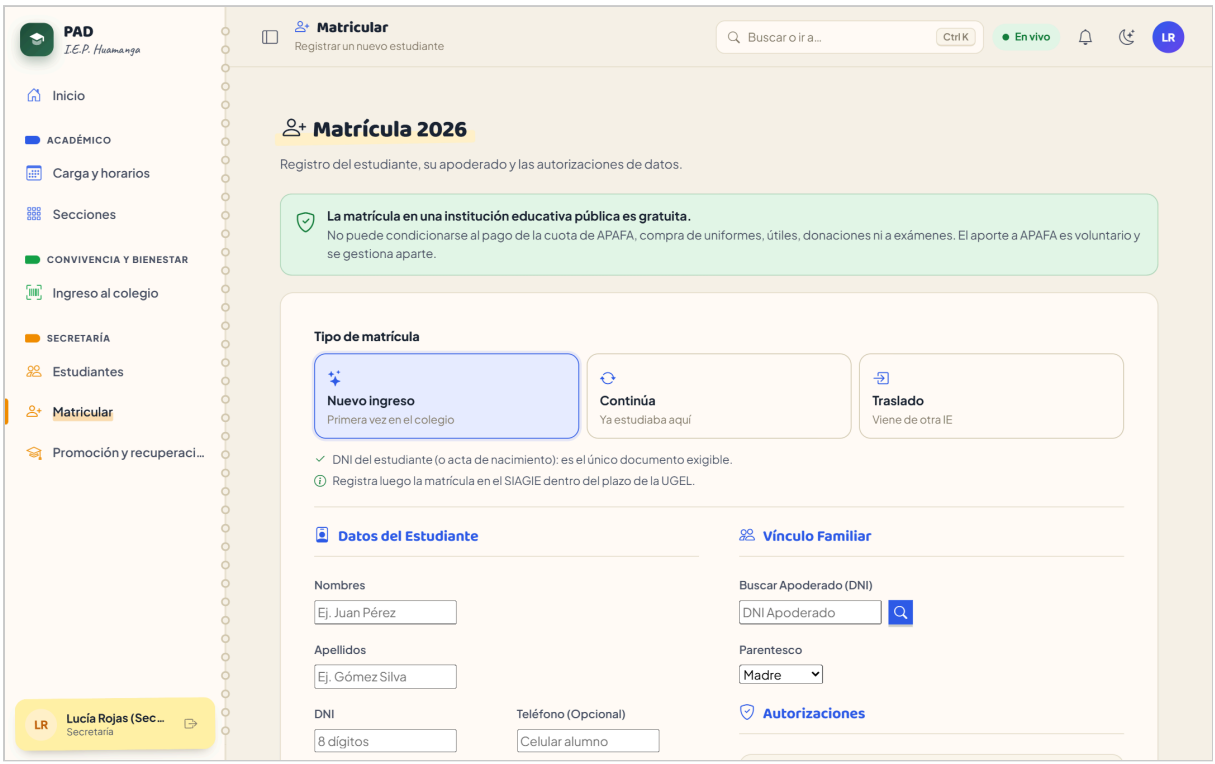




Nota. Las reglas (máximo de periodos seguidos del mismo curso, carga del docente) se respetan al generar; la vista «Por docente» permite revisar la carga de cada profesor. Capturas del sistema en ejecución.

Figura 16

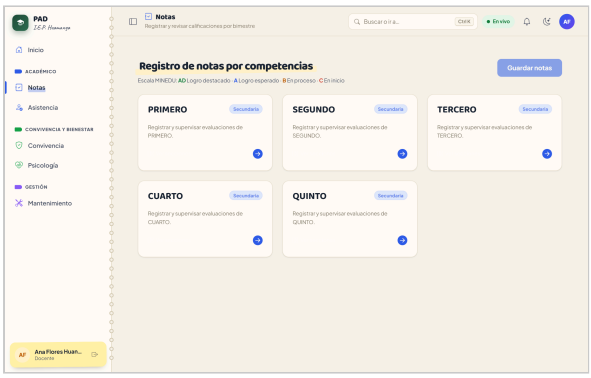
Matrícula 2026 gratuita con tipo de matrícula (Secretaría)

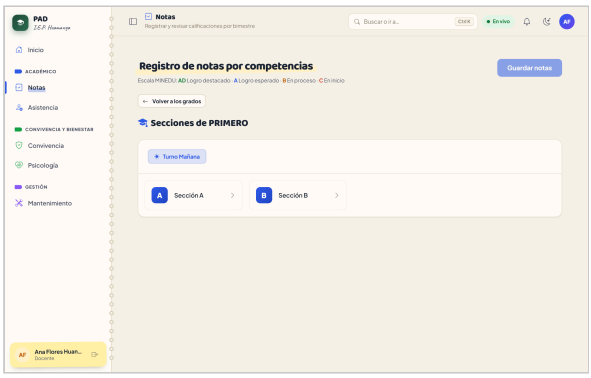


Nota. El formulario recuerda que la matrícula en la IE pública es gratuita, pide solo el DNI o el acta de nacimiento y distingue nuevo ingreso, continuidad y traslado (con IE y código modular de procedencia). Captura del sistema en ejecución.

Figura 17

Registro de notas por competencias: elección de grado y sección (Docente)

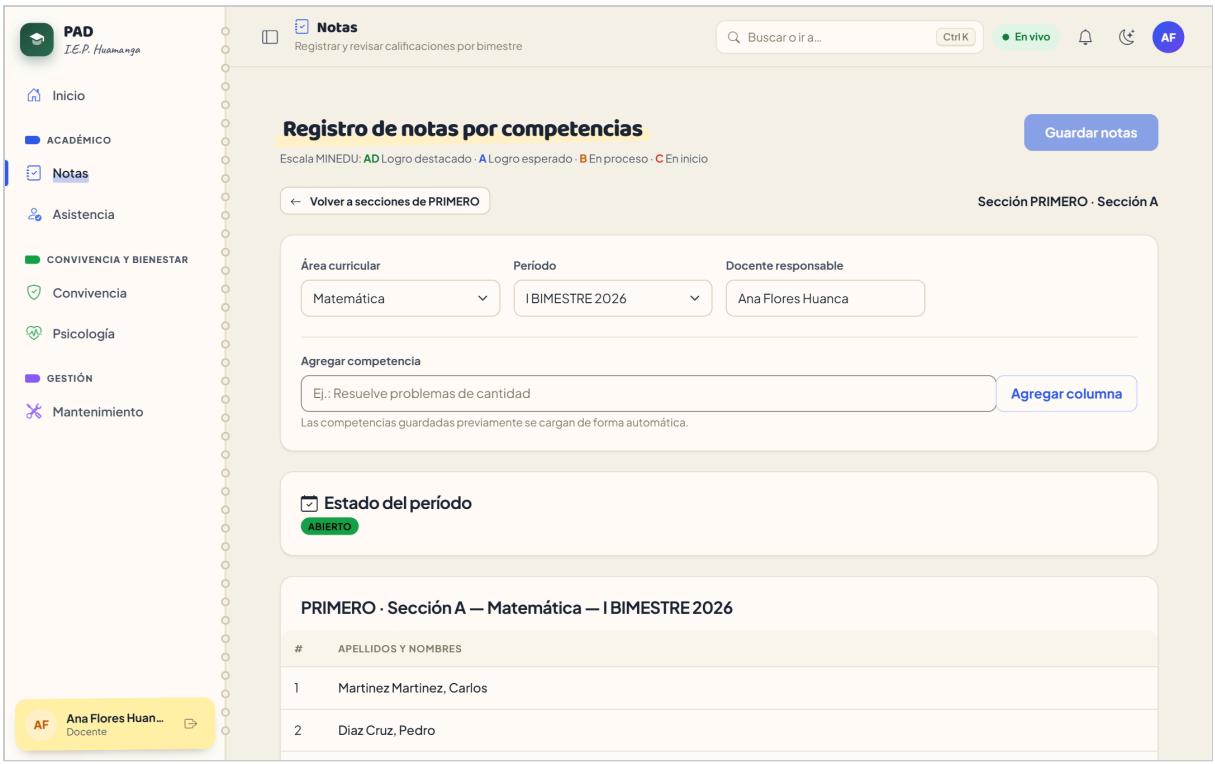




Nota. La escala MINEDU (AD, A, B, C) se muestra siempre en la cabecera. El docente solo ve los grados y secciones de su carga académica. Capturas del sistema en ejecución.

Figura 18

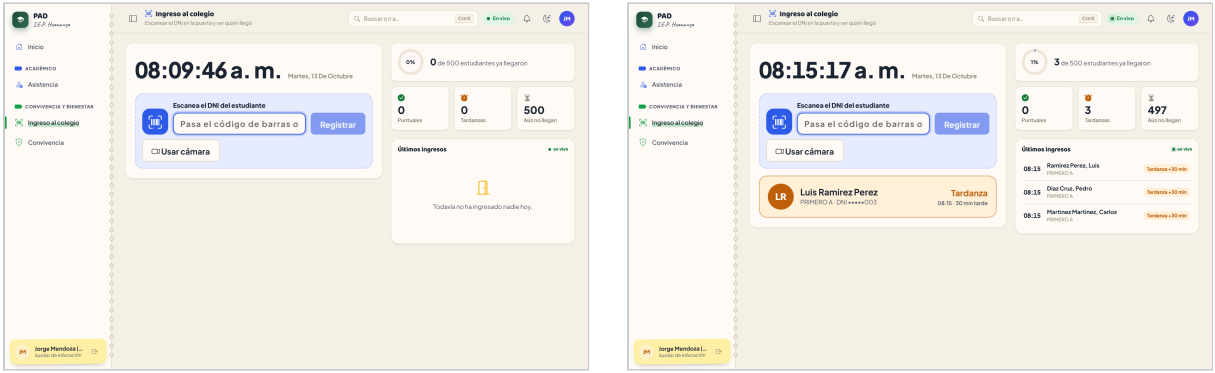
Registro de notas de Matemática – PRIMERO A, I Bimestre 2026



Nota. El docente agrega columnas de competencias y registra la calificación literal de cada estudiante, por teclado o por voz («Carlos, A»). El estado del bimestre (ABIERTO) proviene del servicio Evaluaciones. Captura del sistema en ejecución.

Figura 19

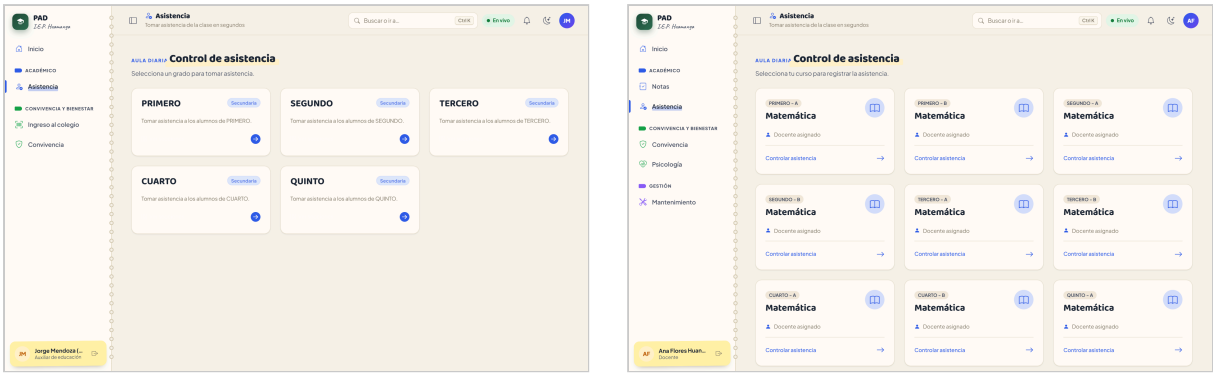
Ingreso al colegio escaneando el DNI (Auxiliar)



Nota. A la izquierda, la pantalla al iniciar el día; a la derecha, después de escanear tres DNI a las 08:15: cada lectura muestra una tarjeta (tardanza de 30 min), actualiza los contadores (3 de 500 llegaron) y la lista «Últimos ingresos» en vivo. El DNI aparece enmascarado. Los registros de prueba se eliminaron después de la captura. Capturas del sistema en ejecución.

Figura 20

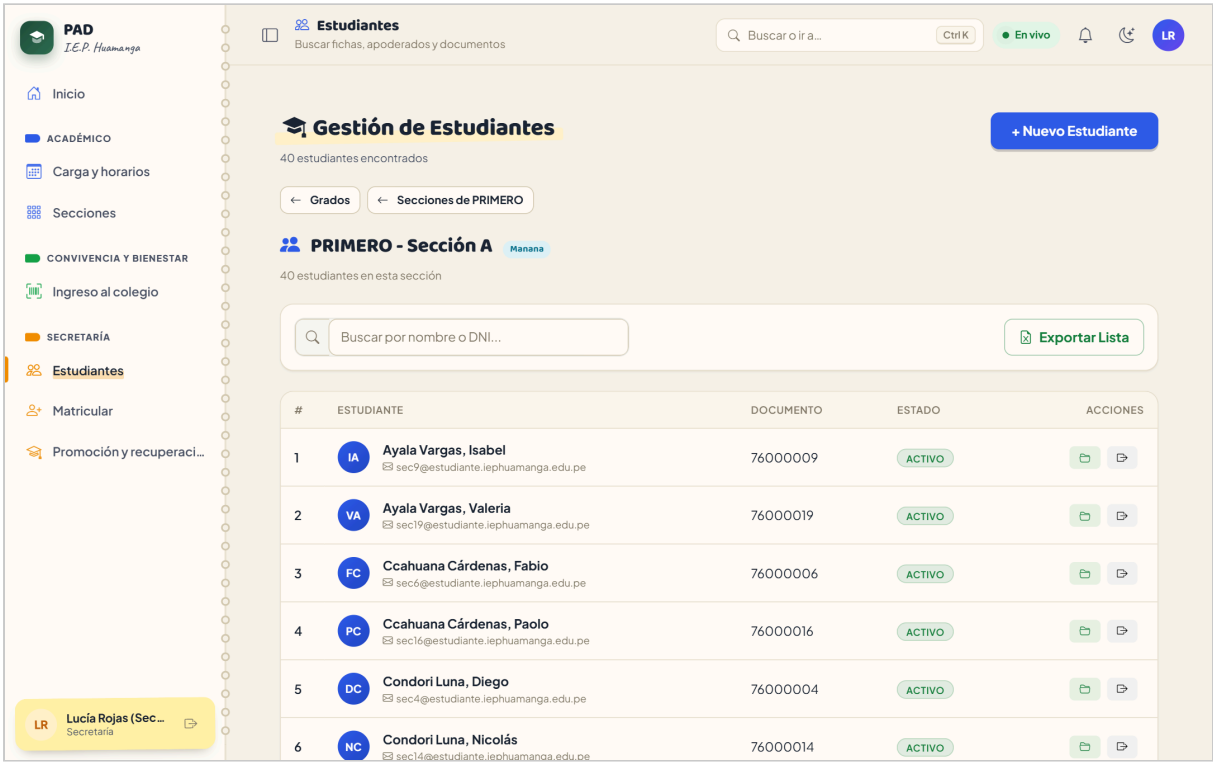
Asistencia (Auxiliar y Docente)



Nota. El flujo es el mismo en todos los módulos: grado → sección → registro, con el atajo «Continuar donde te quedaste». Capturas del sistema en ejecución.

Figura 21

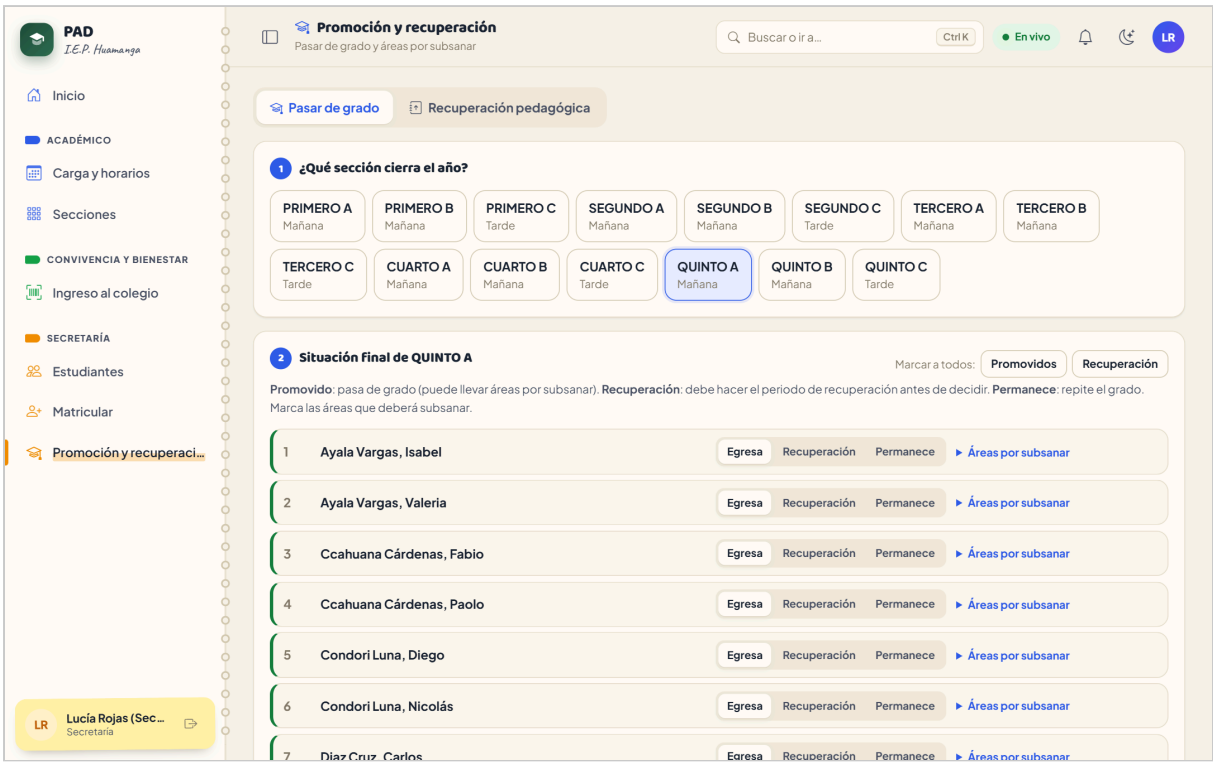
Gestión de estudiantes por sección (Secretaría)



Nota. Lista de PRIMERO A (40 estudiantes activos) con búsqueda por nombre o DNI, exportación, legajo y traslado de salida. Captura del sistema en ejecución.

Figura 22

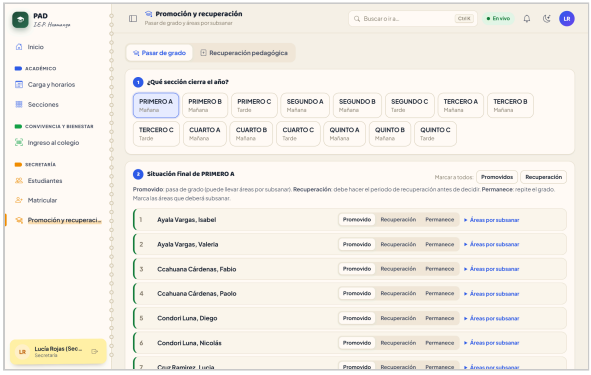
Promoción de grado: situación final de QUINTO A

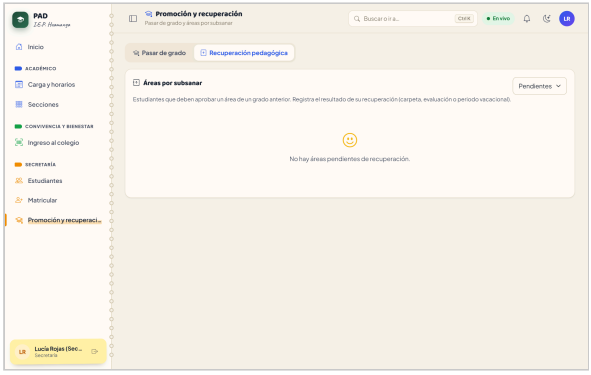


Nota. Para quinto grado la opción de promoción es «Egresa». En los demás grados se elige promovido, recuperación o permanece, y se marcan las áreas por subsanar (RVM N.° 094-2020-MINEDU). Captura del sistema en ejecución.

Figura 23

Promoción de PRIMERO A y recuperación pedagógica

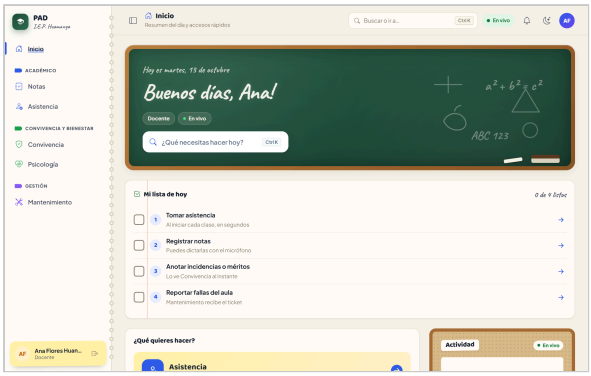


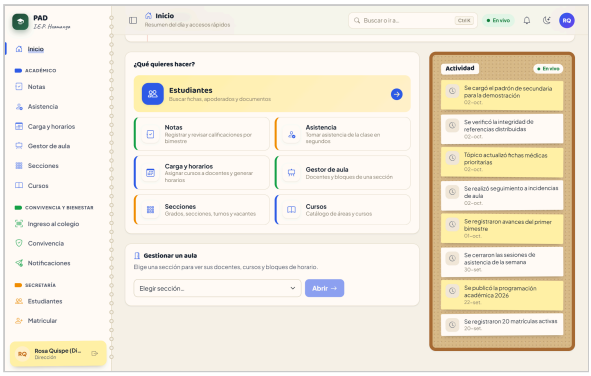


Nota. Capturas del sistema en ejecución.

Figura 24

Inicio del Docente y periódico mural de actividad

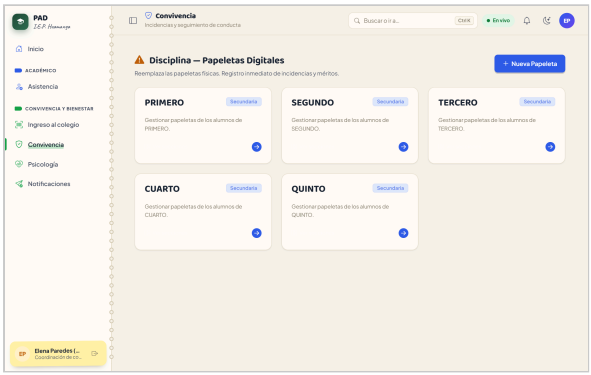


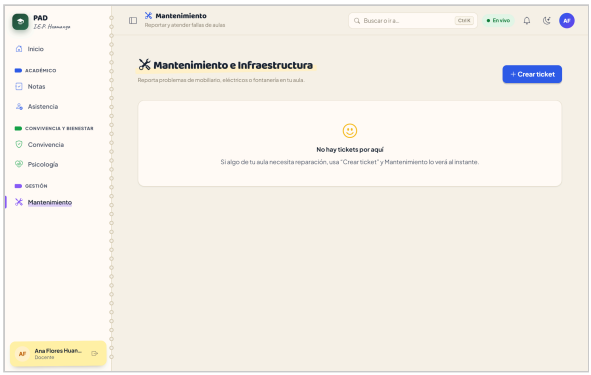


Nota. La lista del docente propone tomar asistencia, registrar notas, anotar incidencias y reportar fallas. Capturas del sistema en ejecución.

Figura 25

Convivencia escolar y mantenimiento





Nota. Capturas del sistema en ejecución.

3.2.2 Código Fuente

La estructura principal del repositorio es la siguiente:

```
sistemaDistribuido/
|-- backend/
|   |-- api-gateway/              (Spring Cloud Gateway, seguridad, tiempo real)
|   |-- servicio-estudiantes/     |-- servicio-cursos/
|   |-- servicio-evaluaciones/    |-- servicio-disciplina/
|   |-- servicio-reportes/        |-- servicio-notificaciones/
|   |-- servicio-horarios/        `-- init-postgres-dbs.sh
|-- frontend/                     (Angular 21: core/, layout/, pages/, shared/)
|-- ops/
|   |-- tests/    (system-smoke, gateway-contract, resilience-smoke, backup-restore)
|   |-- demo/     (datos de secundaria 2026)   |-- observability/  |-- backup/
|-- documentacion/                (auditoría, matriz de roles, pruebas, informe)
|-- docker-compose.yml
`-- .env.example
```

Cada backend se organiza en `config/`, `controller/`, `dto/`, `exception/`, `model/`, `repository/`, `service/` y `resources/db/migration/`. El frontend separa `core/` (servicios, interceptores, guardas, navegación y tiempo real), `layout/` (menú y barra superior), `pages/` (un módulo por pantalla) y `shared/` (componentes reutilizables). Como ejemplo, el filtro global del gateway que publica los cambios en tiempo real:

```
@Component
public class CambiosGlobalFilter implements GlobalFilter, Ordered {
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        HttpMethod metodo = exchange.getRequest().getMethod();
        if (!ESCRITURAS.contains(metodo)) return chain.filter(exchange);
        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            HttpStatusCode estado = exchange.getResponse().getStatusCode();
            if (estado != null && estado.is2xxSuccessful()) {
                tiempoReal.publicar(exchange.getRequest().getPath().value(), metodo,
usuario(exchange));
            }
        }));
    }
}
```

3.2.3 Despliegue

El despliegue completo se ejecuta desde la raíz del proyecto con un solo comando. Docker Compose construye las imágenes, crea la red `pad-network` y los volúmenes, y arranca los servicios en orden: el gateway espera a que todos los microservicios estén sanos y estos esperan a PostgreSQL y RabbitMQ.

```
> copy .env.example .env      (definir DB_PASSWORD, JWT_SECRET, INTERNAL_API_KEY…)
> docker compose up -d --build
> docker compose --profile demo run --rm demo-data      (opcional: datos de demostración)
> docker compose ps
```

La Figura 26 evidencia el estado del despliegue: los once contenedores en ejecución y sanos, el *health check* del gateway y las colas de RabbitMQ.

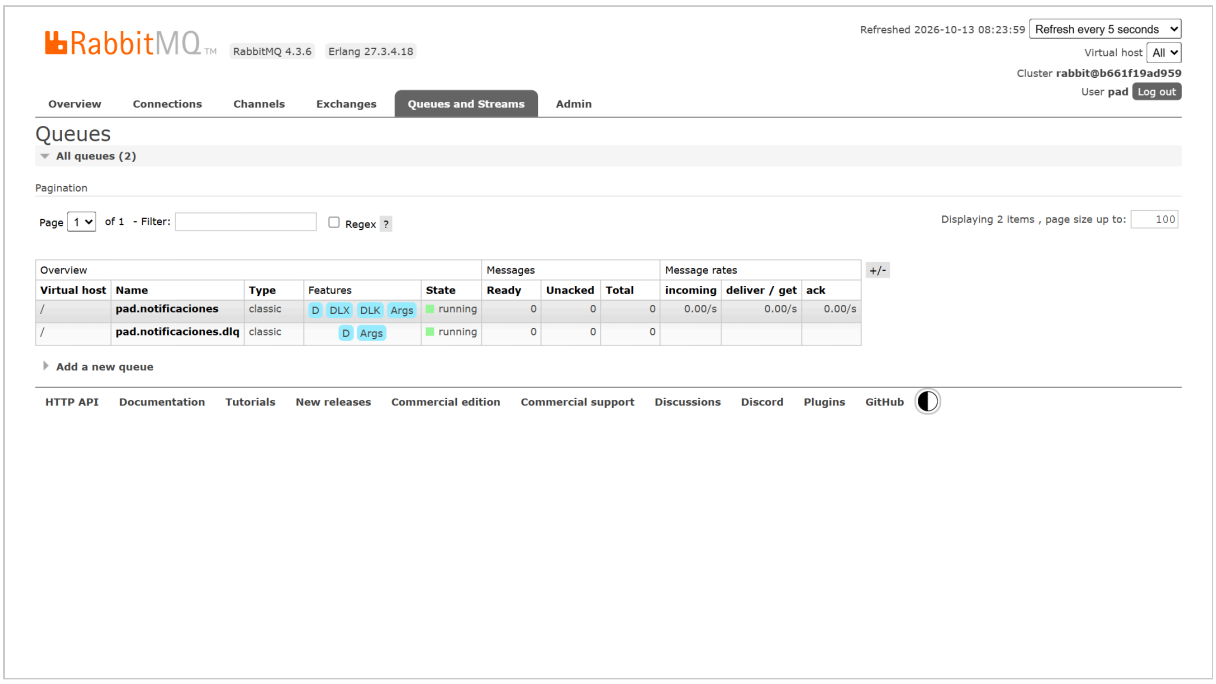
Figura 26

Evidencia del despliegue: contenedores, salud del gateway y colas de RabbitMQ

```
> docker compose ps --format "table {{.Name}}\t{{.Status}}\t{{.Ports}}"
NAME                STATUS                    PORTS
pad-api-gateway     Up 8 hours (healthy)      127.0.0.1:8080->8080/tcp
pad-cursos          Up 8 hours (healthy)      8083/tcp
pad-disciplina      Up 12 hours (healthy)     8081/tcp
pad-estudiantes     Up 8 hours (healthy)      8082/tcp
pad-evaluaciones    Up 12 hours (healthy)     8084/tcp
pad-frontend        Up 31 minutes (healthy)   0.0.0.0:80->80/tcp
pad-horarios        Up 12 hours (healthy)     8088/tcp
pad-notificaciones  Up 12 hours (healthy)     8086/tcp
```

```
pad-postgres          Up 12 hours (healthy)    127.0.0.1:5433->5432/tcp
pad-rabbitmq          Up 12 hours (healthy)    127.0.0.1:5672->5672/tcp, 127.0.0.1:15672-
>15672/tcp
pad-reportes          Up 12 hours (healthy)    8085/tcp

> curl http://localhost:8080/actuator/health
{"status":"UP"}
```



Nota. Consola de RabbitMQ 4.3.6: colas durables `pad.notificaciones` (con DLX/DLK) y `pad.notificaciones.dlq` en estado *running*, sin mensajes atascados. Evidencia obtenida el 13/10/2026.

Para un entorno productivo (por ejemplo, una instancia EC2 de la UGEL o de la institución) deben configurarse además HTTPS, la gestión externa de secretos, las copias de seguridad programadas (perfil `operations`) y la observabilidad (perfil `observability`). La guía `documentacion/GUIA_DEMO_SECUNDARIA_EC2.md` describe ese procedimiento.

4. Cierre del Proyecto

4.1 Pruebas

El sistema se verificó en seis niveles. Todas las pruebas se ejecutaron el 13 de octubre de 2026 sobre el código final; la Tabla 19 resume los resultados y las salidas completas se incluyen en el Anexo B.

1. **Pruebas de integración del backend** (JUnit 5 + Testcontainers): cada servicio levanta un PostgreSQL real en un contenedor, aplica sus migraciones Flyway y prueba controladores, reglas de negocio y seguridad.
2. **Pruebas unitarias del frontend** (Vitest): componentes, servicios, interceptor JWT, servicio de tiempo real, generador de horarios y dictado por voz.
3. **Prueba de humo del sistema** (`ops/tests/system-smoke.ps1`): inicia sesión con cada rol contra el sistema desplegado y verifica 50 accesos permitidos y bloqueados a través del gateway.
4. **Prueba de contrato del gateway** (`ops/tests/gateway-contract.ps1`): comprueba que las rutas lleguen al servicio correcto y que las reglas de cierre y reapertura de bimestres se respeten.
5. **Prueba de resiliencia** (`ops/tests/resilience-smoke.ps1`): detiene el servicio Cursos, comprueba que el gateway responda con un 503 controlado y que el servicio vuelva a atender al reiniciarse.
6. **Prueba de persistencia**: reinicia el contenedor de PostgreSQL y compara los registros antes y después.

Tabla 19

Resumen de resultados de pruebas

Nivel	Herramienta	Alcance	Resultado
Backend	JUnit 5 + Testcontainers	Gateway y 7 microservicios (30 clases de prueba)	**64 / 64 aprobadas**
Frontend	Vitest	18 archivos de prueba	**36 / 36 aprobadas**
Sistema	system-smoke.ps1	Login por rol, accesos permitidos y bloqueados	**50 / 50 verificaciones**
Contrato	gateway-contract.ps1	Rutas del gateway y reglas de bimestres	**10 / 10 verificaciones**

Nivel	Herramienta	Alcance	Resultado
Resiliencia	resilience-smoke.ps1	Caída y recuperación de Cursos vía gateway	**2 / 2 verificaciones**
Persistencia	restart postgres	500 estudiantes y 595 bloques de horario	**Datos intactos**
Despliegue	docker compose ps	Contenedores con *health check*	**11 / 11 sanos**

Nota. Elaboración propia a partir de las salidas de consola del Anexo B.

Algunas pruebas de backend destacadas: `TrayectoriaIngresoIntegrationTest` (matrícula, traslado de salida, promoción con áreas por subsanar, ingreso por DNI con tardanza y rechazo de duplicados), `HorarioLoteIntegrationTest` (guardado atómico de varias secciones y rechazo de cruces y sobrecarga), `RepartoCargaIntegrationTest` (reparto de un curso entre varios docentes), `EstudiantesAuthIntegrationTest` (JWT, cambio obligatorio de clave y roles) y `TiempoRealServiceTest` del gateway (eventos SSE filtrados por audiencia de rol).

4.2 Validación

La validación funcional se realizó recorriendo el sistema con cada rol. La Tabla 20 contrasta cada criterio con su resultado y la evidencia correspondiente.

Tabla 20

Matriz de validación funcional

N.°	Criterio de validación	Resultado	Evidencia
1	Cada rol ve solo sus módulos y las rutas ajenas responden 403.	**Cumple**	Fig. 12, 14, 17, 19; smoke
2	La matrícula es gratuita, exige solo el DNI y distingue nuevo, continúa y traslado.	**Cumple**	Fig. 16
3	El horario se genera con parámetros elegidos y se puede editar y guardar sin cruces.	**Cumple**	Fig. 14, 15; `HorarioLote-IntegrationTest`
4	Un curso puede repartirse entre varios docentes sin exceder sus horas.	**Cumple**	`RepartoCarga-IntegrationTest`
5	El escaneo del DNI registra el ingreso, detecta tardanzas y se refleja en vivo.	**Cumple**	Fig. 19
6	El docente registra notas AD/A/B/C solo en su sección y bimestre abierto.	**Cumple**	Fig. 17, 18; contrato
7	Coordinación cierra el bimestre y solo Dirección lo reabre.	**Cumple**	Contrato (Anexo B)

N.°	Criterio de validación	Resultado	Evidencia
8	La promoción registra la situación final y las áreas por subsanar; quinto grado egresa.	**Cumple**	Fig. 22, 23
9	Las notificaciones pasan por la outbox y RabbitMQ con DLQ.	**Cumple**	Fig. 26
10	Si un servicio cae, el resto sigue funcionando y el gateway responde con un mensaje controlado.	**Cumple**	Anexo B5
11	Los cambios de un usuario aparecen en las pantallas de los demás sin recargar.	**Cumple**	Fig. 12; `TiempoRealServiceTest`
12	Los datos persisten tras reiniciar los contenedores (volúmenes).	**Cumple**	Anexo B6

Nota. Elaboración propia.

4.3 Entrega y Puesta en Marcha

La entrega incluye el repositorio completo, este informe, el archivo `docker-compose.yml`, el archivo `.env.example`, los scripts de prueba y demostración (`ops/`) y la documentación técnica (`documentacion/`: auditoría, matriz de roles, política de datos y recuperación, guía de pruebas y guía de despliegue en EC2). Para la demostración se dispone de un conjunto de datos de secundaria 2026: cinco grados, quince secciones en dos turnos, 500 estudiantes, 30 docentes con su carga académica y horarios cargados, y cuentas de demostración para cada rol, que exigen cambiar la contraseña en el primer ingreso.

La puesta en marcha en producción requiere todavía un dominio con HTTPS, la custodia de secretos fuera del servidor, copias de seguridad programadas y verificadas, la contratación de los proveedores reales de correo, SMS o WhatsApp y la capacitación breve del personal. La carga oficial de matrícula y notas en el SIAGIE se mantiene como obligación de la institución; el sistema prepara la información, pero no se integra directamente con el SIAGIE.

Conclusiones

Desde Sistemas Distribuidos

1. La plataforma funciona como un sistema distribuido real: once procesos independientes se comunican por HTTP y AMQP y se presentan al usuario como una sola aplicación, gracias a la transparencia que aporta el API Gateway.

2. La combinación de límites de tiempo, *circuit breaker*, reintentos con espera exponencial y DLQ evita los fallos en cascada; la prueba de resiliencia demostró que la caída de un servicio no detiene al resto.
3. El patrón Outbox con entrega idempotente sustituye a las transacciones distribuidas: garantiza que ningún aviso se pierda ni se duplique, aceptando consistencia eventual solo donde no afecta la operación.

Desde Arquitectura de Software

4. Dividir el sistema por contextos del dominio escolar produjo servicios cohesivos y poco acoplados, lo que permitió agregar matrícula por tipos, ingreso con DNI, promoción y recuperación sin modificar los demás servicios.
5. Los patrones API Gateway, *Database per Service*, Outbox y *Circuit Breaker*, documentados con las vistas 4+1 y un registro de decisiones, hacen explícitos los compromisos asumidos (por ejemplo, una sola instancia de PostgreSQL a cambio de menor costo).
6. Los atributos de calidad priorizados para un colegio público —seguridad, disponibilidad en la hora de ingreso, usabilidad y mantenibilidad— se tradujeron en tácticas concretas y verificables con 64 pruebas de backend, 36 de frontend, 50 de humo y 10 de contrato.

Desde Gestión de Base de Datos

7. Las siete bases normalizadas, con 55 claves primarias, 32 foráneas, 24 restricciones únicas y 19 CHECK, llevan las reglas del colegio al nivel de la base, de modo que ni un error de código ni dos operaciones simultáneas pueden dejar datos inválidos.
8. Las transacciones ACID, el control optimista y las migraciones Flyway con validación del esquema garantizan integridad y reproducibilidad; la auditoría con Envers y la bitácora de bimestres dan trazabilidad de quién cambió qué.
9. Los respaldos cifrados con restauración verificada y la persistencia comprobada ante reinicios aseguran que la información académica de los estudiantes pueda recuperarse ante un incidente.

Referencias

Angular. (s. f.). *Angular documentation*. https://angular.dev

Docker Inc. (s. f.). *Docker documentation*. https://docs.docker.com

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

International Organization for Standardization. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Product quality model*. ISO.

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42–50. https://doi.org/10.1109/52.469759

Ley N.° 29733, Ley de Protección de Datos Personales. (2011). Congreso de la República del Perú.

Ministerio de Educación del Perú. (2016). *Currículo Nacional de la Educación Básica*. MINEDU.

Ministerio de Educación del Perú. (2020). *Resolución Viceministerial N.° 094-2020-MINEDU: Norma que regula la evaluación de las competencias de los estudiantes de la Educación Básica*. MINEDU.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Brewer, E. (2012). CAP twelve years later: How the "rules" have changed. *Computer, 45*(2), 23–29. https://doi.org/10.1109/MC.2012.37

Broadcom. (s. f.). *RabbitMQ documentation*. https://www.rabbitmq.com/docs

Redgate. (s. f.). *Flyway documentation*. https://documentation.red-gate.com/flyway

Resilience4j. (s. f.). *Resilience4j documentation*. https://resilience4j.readme.io

Richardson, C. (2018). *Microservices patterns*. Manning Publications.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7.ª ed.). McGraw-Hill.

Spring. (s. f.-a). *Spring Boot*. https://spring.io/projects/spring-boot

Spring. (s. f.-b). *Spring Cloud Gateway*. https://spring.io/projects/spring-cloud-gateway

Spring. (s. f.-c). *Spring Security reference*. https://docs.spring.io/spring-security/reference/

Tanenbaum, A. S., & van Steen, M. (2017). *Distributed systems* (3.ª ed.). Maarten van Steen.

Testcontainers. (s. f.). *Testcontainers for Java*. https://java.testcontainers.org

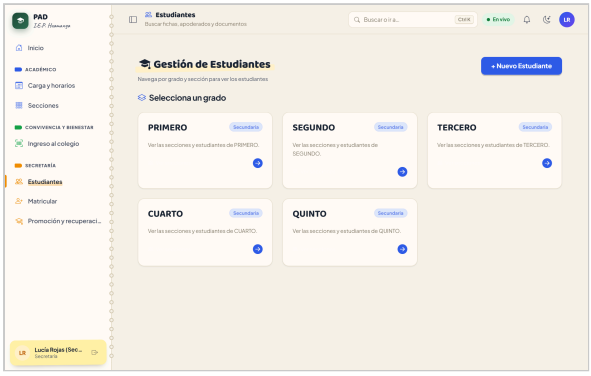
Timefold. (s. f.). *Timefold Solver documentation*. https://docs.timefold.ai

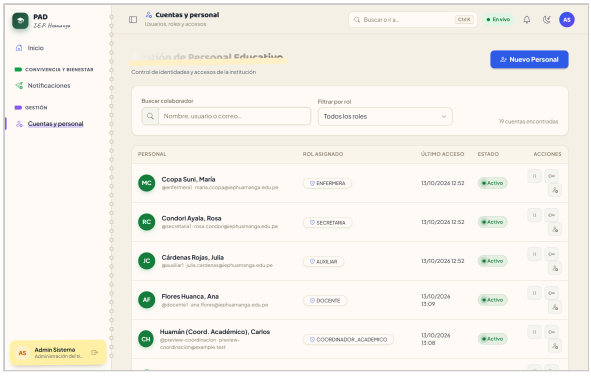
Anexos

Anexo A. Capturas adicionales de la interfaz

Figura A1

Elección de grado en Estudiantes y gestión de usuarios (Administración del sistema)

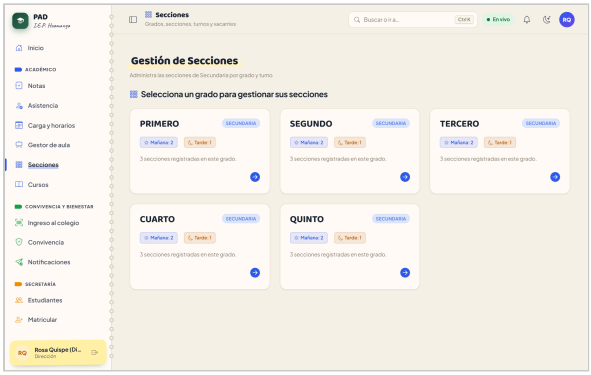


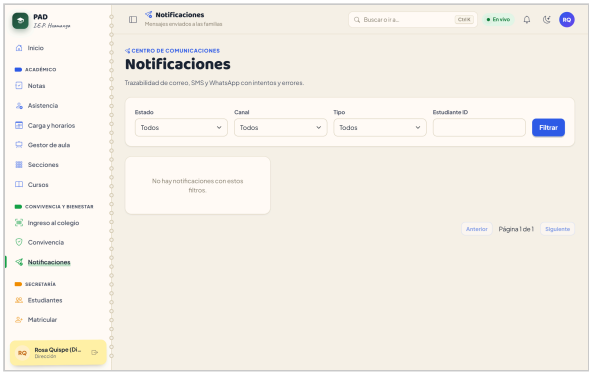


Nota. Capturas del sistema en ejecución.

Figura A2

Secciones y notificaciones (Dirección)

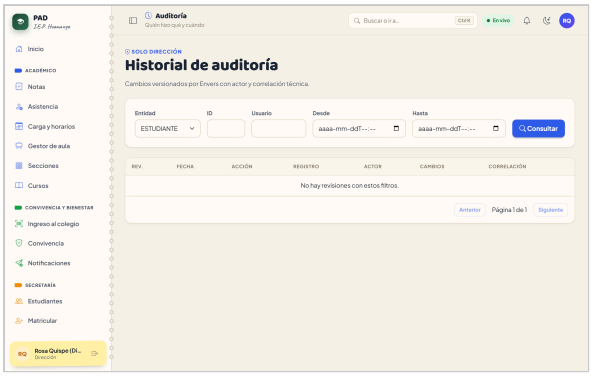




Nota. El centro de comunicaciones permite filtrar por estado, canal, tipo y estudiante. Capturas del sistema en ejecución.

Figura A3

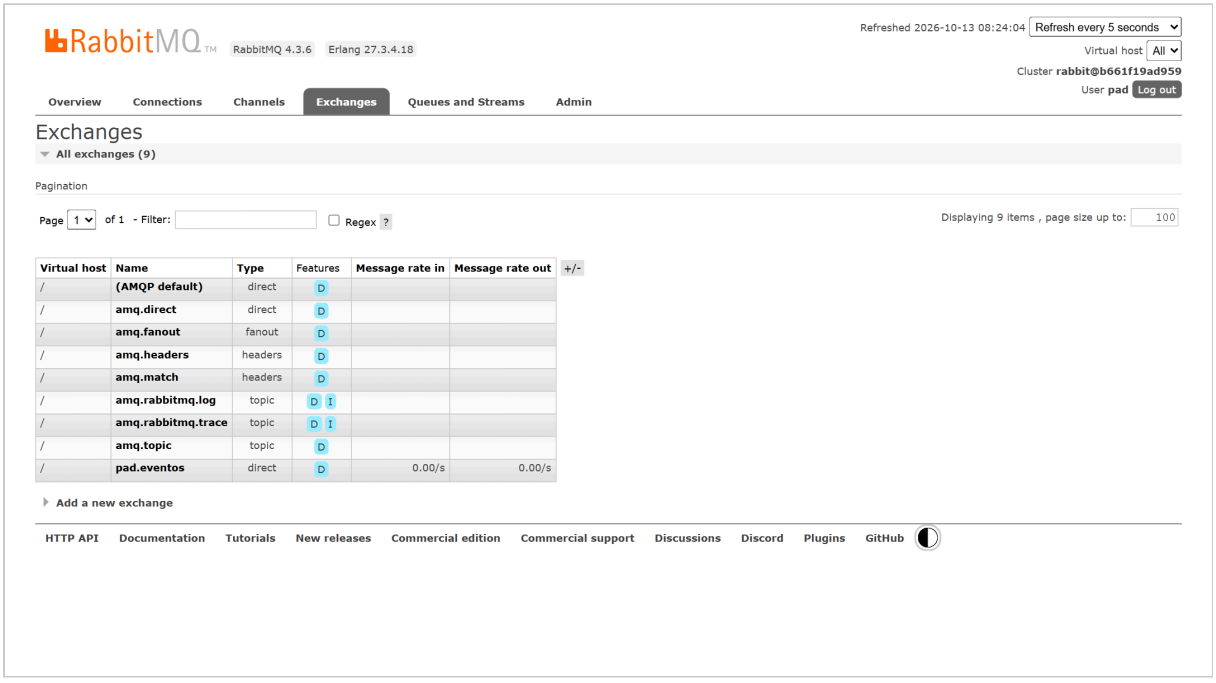
Historial de auditoría y convivencia del docente



Nota. Capturas del sistema en ejecución.

Figura A4

Intercambios de RabbitMQ



Nota. El intercambio directo `pad.eventos` enruta la clave `notificacion.entregar` hacia la cola de notificaciones. Evidencia obtenida el 13/10/2026.

Anexo B. Salidas de consola de las pruebas

B1. Pruebas de integración del backend (Maven + Testcontainers, contenedor `maven:3.9.11-eclipse-temurin-17`):

```
===== api-gateway
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- GatewayContractIntegrationTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- TiempoRealServiceTest
===== servicio-estudiantes
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 -- EstudiantesAuthIntegrationTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- TrayectoriaIngresoIntegrationTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- DocumentoServiceTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- FichaMedicaServiceTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- UsuarioServiceTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- SistemaAcademicoApplicationTests
===== servicio-cursos
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- AsignacionAcademicaIntegrationTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- HorarioLoteIntegrationTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- RepartoCargaIntegrationTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- HorarioServiceTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- SistemaAcademicoApplicationTests
===== servicio-evaluaciones
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- AsistenciaIntegrationTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- BimestreIntegrationTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- DocenteScopeIntegrationTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- NotaServiceTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- SistemaAcademicoApplicationTests
===== servicio-disciplina
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- DisciplinaOutboxIntegrationTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- ReporteDisciplinarioServiceTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- SistemaAcademicoApplicationTests
===== servicio-reportes
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- ActividadControllerTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- ReportesIntegrationTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- ReconciliacionIntegridadServiceTest
===== servicio-notificaciones
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- NotificacionesIntegrationTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 -- NotificacionServiceTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 -- ProveedorNotificacionTest
===== servicio-horarios
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- SistemaAcademicoApplicationTests
TOTAL: 64 pruebas, 0 fallos, 0 errores — BUILD SUCCESS en los 8 proyectos
```

B2. Pruebas unitarias del frontend (`npx ng test --watch=false`):

```
 Test Files  18 passed (18)
      Tests  36 passed (36)
   Start at  08:23:28
   Duration  6.12s (transform 3.06s, setup 6.74s, import 5.10s, tests 5.20s, environment
45.48s)
```

B3. Prueba de humo del sistema (`ops/tests/system-smoke.ps1 -UseDemoCredentials`):

```
[OK] Gateway sin token (401)
[OK] Login admin
[OK] Sesión actual (200)
```

```
[OK] Usuarios paginados (200)
[OK] Trazabilidad de notificaciones (200)
[OK] Incidencias de integridad (200)
[OK] Login coord.academica
[OK] Cursos (200)
[OK] Docentes (200)
[OK] Secciones (200)
[OK] Carga académica (200)
[OK] Notas paginadas (200)
[OK] Estado de bimestres para coordinación (200)
[OK] Auditoría de bimestre para coordinación (200)
[OK] Login secretaria1
[OK] Matrículas para secretaría (200)
[OK] Cursos para secretaría (200)
[OK] Docentes para secretaría (200)
[OK] Secciones para secretaría (200)
[OK] Carga para secretaría (200)
[OK] Horario por sección (200)
[OK] Historial de consentimientos (200)
[OK] Login docente1
[OK] Notas por docente (200)
[OK] Asistencia por docente (200)
[OK] Disciplina por docente (200)
[OK] Tickets por docente (200)
[OK] Derivaciones por docente (200)
[OK] Estado de bimestres para docente (200)
[OK] Login coord.convivencia
[OK] Asistencia convivencia (200)
[OK] Disciplina convivencia (200)
[OK] Derivaciones convivencia (200)
[OK] Login auxiliar1
[OK] Asistencia auxiliar (200)
[OK] Disciplina auxiliar (200)
[OK] Login psicologa1
[OK] Derivaciones psicología (200)
[OK] Login enfermera1
[OK] Fichas médicas (200)
[OK] Atenciones médicas (200)
[OK] Login mantenimiento1
[OK] Bandeja de mantenimiento (200)
[OK] Aulas para mantenimiento (200)
[OK] Login apoderado1
[OK] Portal de apoderado (200)
[OK] Docente bloqueado en usuarios (403)
[OK] Docente bloqueado en auditoría de bimestre (403)
[OK] Administrador bloqueado en estudiantes (403)
Smoke test aprobado: 50 verificaciones.
```

B4. Prueba de contrato del gateway (`ops/tests/gateway-contract.ps1`):

```
[OK] Docente consulta notas con alcance vacío (200)
[OK] Docente consulta estado de bimestres (200)
[OK] Docente no consulta auditoría de bimestres (403)
[OK] Coordinación cierra bimestre (200)
[OK] Coordinación no reabre bimestre (403)
[OK] Dirección reabre bimestre (200)
[OK] Dirección consulta auditoría completa (200)
[OK] Secretaría consulta consentimientos (200)
```

```
[OK] Contrato POST de consentimiento llega al servicio (400)
Prueba contractual aprobada: 10 verificaciones.
```

B5. Prueba de resiliencia (`ops/tests/resilience-smoke.ps1 -UseDemoCredentials`):

```
 Container pad-cursos Stopped
[OK] Caída controlada observada mediante HTTP 503.
 Container pad-postgres Waiting
 Container pad-postgres Healthy
 Container pad-cursos Starting
 Container pad-cursos Started
[OK] El servicio volvió a responder a través del gateway.
```

B6. Prueba de persistencia (reinicio del contenedor PostgreSQL):

```
> antes del reinicio
estudiantes=500
horarios=595
> docker compose restart postgres
 Container pad-postgres Restarting
 Container pad-postgres Started
> después del reinicio
estudiantes=500
horarios=595
```